

深圳大学考试答题纸

(以论文、报告等形式考核专用)

二〇二二 ~ 二〇二四 学年度第 二 学期

课程编号	1501990018	课序号	2	课程名称	机器学习	主讲教师	张书瑜	评分	
学号	2022152021	姓名	王培鸿	专业年级	计算机科学与技术 (卓越班)				

教师评语:

题目:

基于 Cycle GAN 的莫奈风格转移

摘要:

随着人工智能、深度学习的迅速发展,基于计算机的智能转换图像风格的需求在抖音、快手、Titok 等短视频平台上日益剧增,用户可以通过两种图像将其中一种图像快速转换为另外一种图像是一件对用户极具吸引力的事情。图像到图像的转换是一类视觉和图形问题,是使用对齐图像对的训练集来学习输入图像和输出图像之间的映射,旨在将一幅图像的风格特征映射到另一幅图像上,同时保持内容特征不变;然而,对于许多任务,配对的训练数据将不可用,在实际应用中用户输入的图像大多都是无匹配图像;因此,本文使用 Cycle GAN 网络实现将非对齐的图像进行风格迁移。

Cycle GAN 是一种无监督的学习框架,可以学习两个不同域之间的映射关系,而无需成对的训练数据。利用 Cycle GAN 的强大能力,学习从真实照片域到莫奈绘画域的转换函数。通过在大量的照片和莫奈绘画数据上训练 Cycle GAN 模型,发现该模型能够很好地捕捉到莫奈风格的特征,并将其成功地迁移到输入的照片上。本文提出了两种 Cycle GAN 模型:第一种是轻量化的 Cycle GAN 模型,第二种是加入残差块的 Cycle GAN 模型。

本文中 Cycle GAN 模型通过对输入图像依次进行下上采样操作,使用两个判别器依次判断生成图像的准确性,两个生成器实现两种图像作用域的相互转换和重建,应用损失函数梯度的反向传播更新参数;增加感知损失和残差块,提高模型的学习能力同时降低生成器生成图片的色调损失,最终获得转换效果较好、具有泛化性的模型。

实验结果表明,我们的方法不仅能够生成逼真的"莫奈风格"图像,而且保留了原始照片的内容结构。我们的方法在各种类型的照片上都表现出色,证明了其在实际应用中的潜力。未来我们还计划将该方法扩展到其他艺术风格的转换,为图像创作提供更加丰富的选择。

关键词: Cycle GAN; 图像风格转换; 感知损失; 生成器; 判别器

深圳大学课程项目报告

课程名称: 机器学习

项目名称: 基于 Cycle GAN 的莫奈风格转移

学 院: 计算机与软件学院

专 业: 计算机科学与技术（卓越班）

任课教师: 张书瑜

报 告 人: 王培鸿 学号: 2022152021

提交时间: 2024.6.21

教 务 处 制

1 题目

基于 Cycle GAN 的莫奈风格转移

2 摘要

随着人工智能、深度学习的迅速发展,基于计算机的智能转换图像风格的需求在抖音、快手、Titok 等短视频平台上日益剧增,用户可以通过两种图像将其中一种图像快速转换为另外一种图像是一件对用户极具吸引力的事情。图像到图像的转换是一类视觉和图形问题,是使用对齐图像对的训练集来学习输入图像和输出图像之间的映射,旨在将一幅图像的风格特征映射到另一幅图像上,同时保持内容特征不变;然而,对于许多任务,配对的训练数据将不可用,在实际应用中用户输入的图像大多都是无匹配图像;因此,本文使用 Cycle GAN 网络实现将非对齐的图像进行风格迁移。

Cycle GAN 是一种无监督的学习框架,可以学习两个不同域之间的映射关系,而无需成对的训练数据。利用 Cycle GAN 的强大能力,学习从真实照片域到莫奈绘画域的转换函数。通过在大量的照片和莫奈绘画数据上训练 Cycle GAN 模型,发现该模型能够很好地捕捉到莫奈风格的特征,并将其成功地迁移到输入的照片上。本文提出了两种 Cycle GAN 模型:第一种是轻量化的 Cycle GAN 模型,第二种是加入残差块的 Cycle GAN 模型。

本文中 Cycle GAN 模型通过对输入图像依次进行下上采样操作,使用两个判别器依次判断生成图像的准确性,两个生成器实现两种图像作用域的相互转换和重建,应用损失函数梯度的反向传播更新参数;增加感知损失和残差块,提高模型的学习能力同时降低生成器生成图片的色调损失,最终获得转换效果较好、具有泛化性的模型。

实验结果表明,我们的方法不仅能够生成逼真的"莫奈风格"图像,而且保留了原始照片的内容结构。我们的方法在各种类型的照片上都表现出色,证明了其在实际应用中的潜力。未来我们还计划将该方法扩展到其他艺术风格的转换,为图像创作提供更加丰富的选择。

关键词: Cycle GAN; 图像风格转换; 感知损失; 生成器; 判别器

3 介绍

深度学习在许多领域都取得了较为突出的成就。其中,计算机视觉领域的应用最为广泛,如基于计算机视觉的发展和需要,产生了行人检测和图像处理(如图像分离)、图像风格转换、图像恢复等诸多图像处理的要求,如手机相机应用的美容功能和面部置换功能等技术也在逐渐融入了人们的生活。

为了在日常手机应用中使用计算机视觉再现一些现实生活中的真实场景,基于生成对抗网络是当前图像风格转换应用中的主导技术。生成式对抗网络可以训练多种类型的生成式模型,而不需要限制要求网络需具备某特性的功能或形式。

本文讨论了利用轻量化的 Cycle GAN 模型以及加入感知损失的 Cycle GAN 模型来用于实现高质量图像风格转换的生成对抗网络。近些年来,通过深度学习、人工智能可以提取有关图像外部表象和潜在特征的信息,这为图像转换提供了更多的可能性和能力,为人工智能的大规模落地创造了技术基础。

3.1 相关工作

Cycle GAN 网络简介

在传统的 GAN 中,将一种图像转换为另一种图像需要使用成对的图像进行训练[2]。Cycle GAN 是特殊的 GAN,它可以将图像从一个域转换到另一个域,而不需要成对的图像来进行训练[1]。大多数 GAN 只训练一个生成网络,而 CycleGAN 则训练两个生成网络和两个判别网络;CycleGAN 是一种无监督的 **image to image model**,它可以在没有成对训练数据的情况下,进行图像的风格转换。Cycle GAN 的原理示意图如图 3.1 所示。

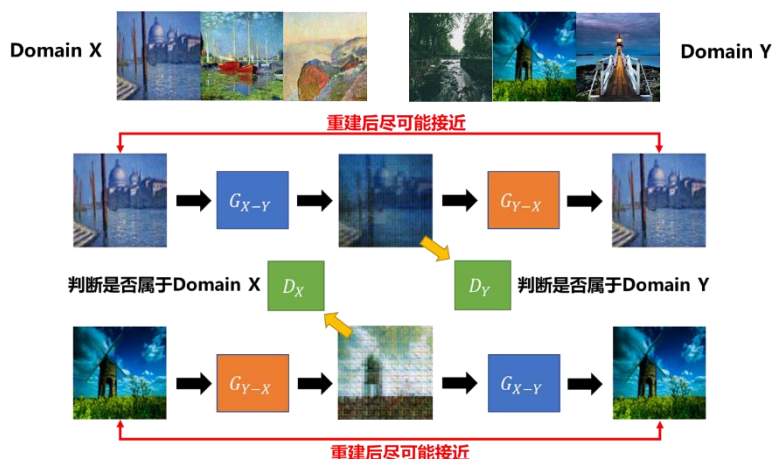


图 3.1 Cycle GAN 原理示意图

如图 1 所示, Cycle GAN 中的输入图像的作用域为 X 和 Y 的非对齐的图像(X 代表莫奈风格图像, Y 代表真实图像), Cycle GAN 网络由两个生成器 $G_{X \rightarrow Y}$ 和 $G_{Y \rightarrow X}$ 、两个判别器 D_X 和 D_Y 组成。网络中同时进行两种风格的相互转换,在第一种转换中,莫奈图像(X 域)通过生成器 $G_{X \rightarrow Y}$ 生成的虚假图像(Y 域)的图像并输入到判别器 D_Y 判断该图像是否属于 Y 域;在为了防止生成器 $G_{X \rightarrow Y}$ 通过使用 Y 域真实图像欺骗判别器 D_Y ,需要将生成的 Y 域虚假图像通过生成器 $G_{Y \rightarrow X}$ 重建回莫奈风格图像(该图像一般称为循环图像),通过尽量减少循环图像与真实莫奈图像的差异,使得生成器 $G_{X \rightarrow Y}$ 和 $G_{Y \rightarrow X}$ 能够更加准确地提取图像的特征进行风格转换。同理,对于第二种转换, Cycle GAN 中先将真实图像转换为虚假莫奈风格图像,再重建回真实风格图像(循环图像),比较循环图像和真实图像的差异作为重建损失,不断训练模型,更新参数。

3.2 本文的研究内容

本文主要针对 Cycle GAN 模型进行了研究,研究发现, Cycle GAN 模型的设计主要分为两个模块(1)生成器;(2)判别器。本文在基于 Tensor flow 框架下以及 kaggle 网站提供的数据集下,实现了一种轻量化的 Cycle GAN 模型和加入残差块的 Cycle GAN 模型。

在轻量化的 Cycle GAN 模型中,本文中使用了尺寸为(128, 128)的图像作为输入,简化了生成器和判别器采取特征的深度,并使用 MobileNetV2 模型计算感知损失,使得模型训练的速度加快并一定程度上保持了生成图片与原图片的色调成分;主要难点在于模型的训练不稳定,收敛速度过快。

在加入残差块的 Cycle GAN 模型中,本文改用了 Zhu, J.-Y[1]等前辈使用的尺寸为(256, 256)的图像输入,主要的工作是加深了生成器、判别器的深度,在生成器中加入残差块,并使用 MobileNetV2[3]模型计算感知损失;使得模型的学习能力更强,生成与原图保持一致的图片色调。

3.3 本文的主要结构

本文注重于基于 Cycle GAN 的莫奈风格转移实现,并在 Cycle GAN 的实现中进行了改进:轻量化、使用 MobileNetV2 计算感知损失。

本文一共包括以下几个部分:

第一部分，介绍了提出了 Cycle GAN 在风格转移的相关工作，并提出了本文的主要内容。

第二部分，介绍轻量化的 Cycle GAN 模型，从模型的架构出发，逐步介绍模型中下采样、上采样、生成器、判别器部分的构成，损失函数的计算原理；并对有无感知损失的轻量化模型进行了比较。

第三部分，介绍加入残差块的 Cycle GAN 模型，同样从系统架构出发，阐述残差块的原理及实现，详细介绍改变图片尺寸后的生成器、判别器的构成，并加入了 MobileNetV2 计算感知损失进行风格转换实验。

第四部分，对全文内容进行简要总结，并提出了当前工作的局限性以及未来的改进工作。

4 轻量化的 Cycle GAN 模型

本章节主要介绍第一种轻量化的 Cycle GAN 模型，即通过两个图像域的生成器循环提取图像的语义特征，生成最大化欺骗判别器的图像；利用两个判别器进行判断输入的图像是否属于对应的图像域，对生成器生成能力进行打分；为尽量减少 Cycle GAN 模型生成图片的色调改变，本文在生成器中添加了轻量化的预训练 MobileNetV2 模型进行感知损失的计算，使得生成的图片细节更丰富、风格更自然。

4.1 总体模型

轻量化的 Cycle GAN 模型提出了一个艺术风格转换的模型，该图像艺术风格化模型由生成器和判别器构成。在模型训练学习过程中,将生成器生成的图片输入 MobileNetV2 中计算感知损失进行联合后输出图片，可降低模型学习时的误差而且使得生成的图像细节和质感更加自然。在 Loss 中添加了感知损失函数，使用轻量化的 MobileNetV2 模型进行计算，在保持内容结构的同时，生成的图像接近于风格化的图像。风格转换模型如下图 4.1 所示。

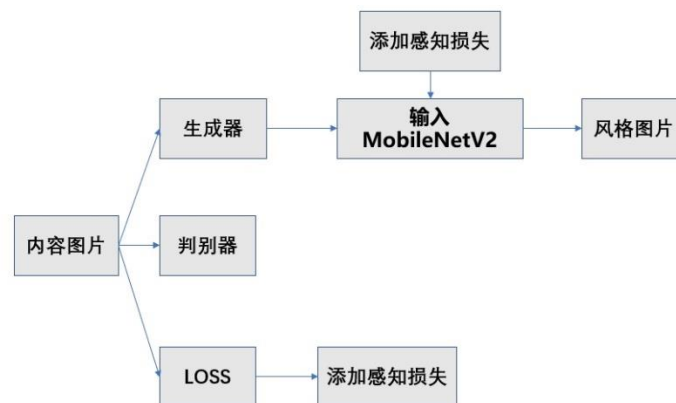


图 4.1 风格转换模型

4.2 相关技术理论

对整体模型结构提出改进之后，下面对模型相关的技术进行学习探究，分别对 Cycle GAN 基础模型、MobileNetV2 模型以及感知损失函数进行了相关研究与学习。

4.2.1 循环一致性生成对抗网络模型

在图像风格转换领域，生成对抗网络 GAN 得到了广泛应用；但大多数的如 WGAN、DCGAN、CGAN 以及 Pix2Pix 等 GAN 网络需要提供对齐、成对的训练数据集；为了解决这个问题，Cycle GAN 实现了可以使用非对齐的数据集进行训练、降低了对数据集的要求[2]。

如图 4.2 所示，Cycle GAN 由两组 GAN 组成，X 和 Y 则代表来自不同域的图像[1]。图像风

格转换任务可以看作是不成对的 X （或 Y ）域的图片到 Y （或 X ）域图片的映射；Cycle GAN 网络结构定义了两个生成器，分别是 $G:X \rightarrow Y$ 和 $F:Y \rightarrow X$ （如图 4.2.a 所示）。两个生成器是循环一致的，即： $F(G(x)) \rightarrow x$ 且 $G(F(y)) \rightarrow y$ 。

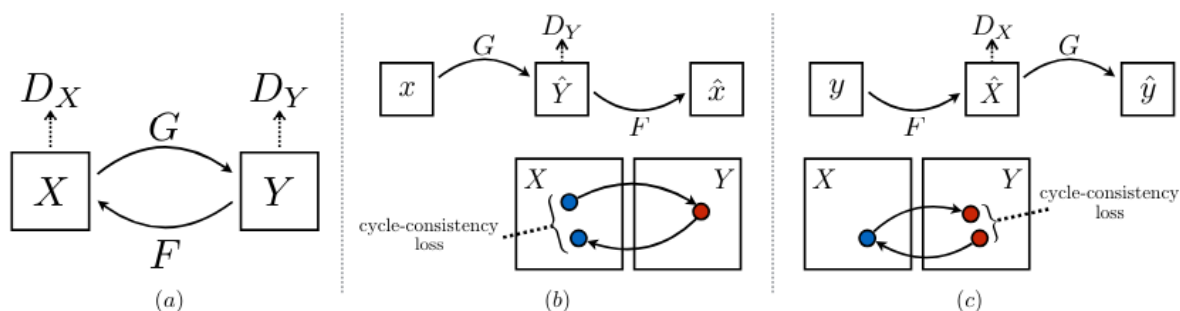


图 4.2 Cycle GAN 原理图

如图 4.2 b 和 c 所示，Cycle GAN 网络中还包含两个判别器 D_x 和 D_y ，依次用于判断输入到判别器的图像是否来自于 X 域、 Y 域。为生成器 G 和 F 过于偏向于欺骗生成器 D 而直接生成与对应作用域相同的图像给判别器 D ，网络中还需要进行重建操作，使得重建后的 x 和 $\hat{x}$ 、 y 和 $\hat{y}$ 图像之间应该尽可能接近，两者的差异/距离称之为 Cycle-consistency Loss（即循环一致性损失）。

4.2.2 MobileNetV2 模型

MobileNetV2 是在 MobileNetV1 基础上提出的 CNN 架构，主要由深度可分离卷积和倒残差结构组成。如下图所示，倒残差结构采用先升维后降维的设计，它能够有效地捕捉更丰富的特征信息。其中，该结构由两个深度可分离卷积组成，第一个深度可分离卷积用于增加通道数，第二个深度可分离卷积用于恢复通道数。MobileNetV2 网络模型中的倒残差结构呈现两头小中间大的形式，因此，模型最后输出的是一个低维的特征信息。为了避免大量丢失低维特征信息，模型在最后一个卷积层采用线性激活函数。

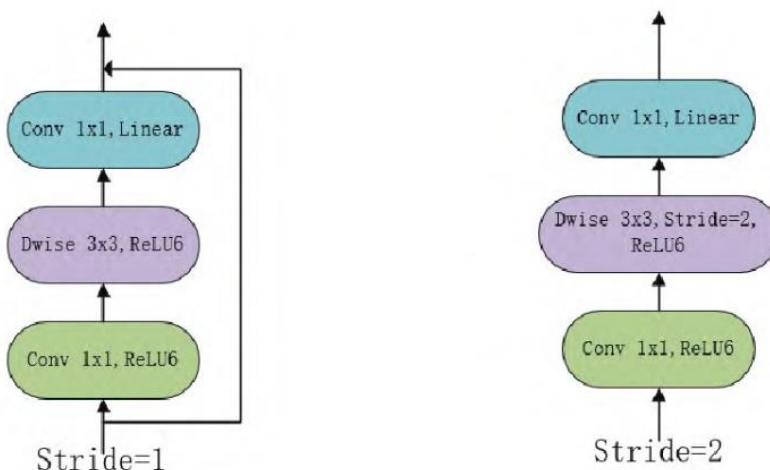


图 4.3 MobileNetV2 模型结构示意图

4.2.3 感知损失函数

感知损失 (Perceptual Loss) 是一种基于深度学习的图像风格迁移方法中常用的损失函数。与传统的均方误差损失函数 (Mean Square Error, MSE) 相比，感知损失更注重图像的感知质量，更符合人眼对图像质量的感受。

本文中感知损失的计算方式通过将输入图像和目标图像分别输入预训练的 MobileNetV2 神经网络，进行特征提取。然后将这些特征表示作为损失函数的输入，计算它们之间的 L1 loss 作为感知损失。感知损失的目标是最小化输入图像和目标图像在特征空间的距离。感知损失可以用于各种图像处理任务中，如图像超分辨率、图像去噪、图像修复、图像风格迁移等。感知损失的计算公式如下：

$$Perceptual\ Loss = \frac{1}{N} |Fi(x) - Fi(y)|$$

其中, x 是输入图像, y 是目标图像, $Fi(x)$ 和 $Fi(y)$ 分别表示它们在 MobileNetV2 神经网络中的第 i 层的特征表示, N 是特征数目。

4.2.4 Dropout 层

Dropout 是一个简单又极其有效的正则化方法, 能够缓解卷积神经网络 CNN 过拟合的现象。

Dropout 叫做随机失活, 简单来说就是在模型训练阶段的前向传播过程中, 让某些神经元的激活值以一定的概率停止工作, 但一般不用于模型测试阶段。如下图所示, 这样可以使模型的泛化性更强。

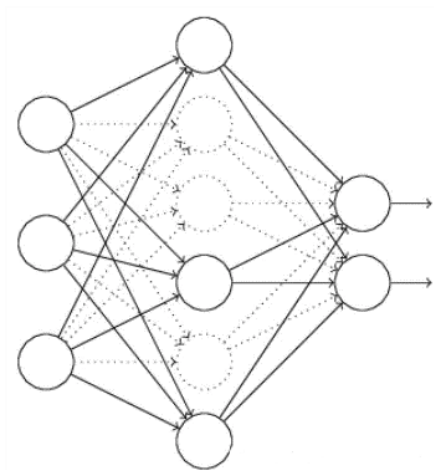


图 4.4 Dropout 原理示意图

因为 Dropout 会随机使神经元失活, 导致两个神经元不一定每次都在一个 dropout 网络中出现, 使得权重的更新不再依赖于有固定关系的节点的共同作用, 迫使网络去学习更加具有泛化性的特征、减少对神经元的过高权重, 从而提高网络的泛化能力。

4.3 模型实现

4.3.1 下采样与上采样

本文中设计的轻量化的 Cycle GAN 网络的下采样部分由 Conv2D 二维卷积层、Batch Normalization 层、Leaky ReLU 激活层组成。下采样部分首先通过均值为 0、标准差为 0.02 的标准正态分布随机初始化 initializer 和 gamma 参数, 使用大小为 (2, 2) 的卷积核、选取卷积步长为 2 的 Conv2D 卷积层保证输入和输出的图片尺寸一致; 最后, 将卷积结果输入到 Batch Normalization 层进行归一化后放到 Leaky ReLU 激活层进行非线性变换, 帮助模型学习到更加丰富的特征表示, 从而提高生成效果。下采样的目标是为了缩小图像并提取特征, 防止过拟合。如下图所示, 依次展示了 PCA 保留一个主成分后重建图像、滤波器数量为 32、64、128、256 的效果图, 可以看到随着下采样的深度增加, 采集的图像特征更加细致。

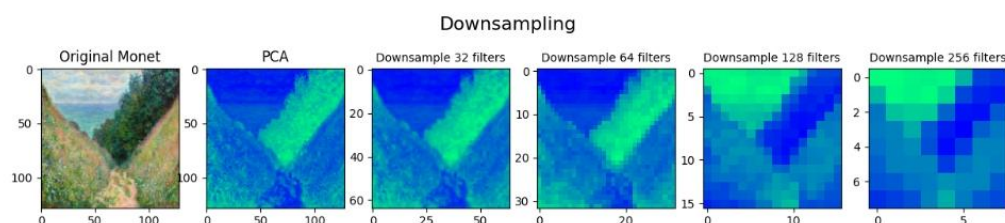


图 4.5 不同深度的下采样示意图

本文中设计的轻量化的 Cycle GAN 网络的上采样部分由 Conv2DTranspose 转置卷积层、Batch Normalization 层、Dropout 层、Leaky ReLU 激活层组成。上采样部分同样首先通过均值为 0、标准差为 0.02 的标准正态分布随机初始化 initializer 和 gamma 参数，使用大小为 (2, 2) 的卷积核、选取卷积步长为 2 的 Conv2DTranspose 转置卷积层保证输入和输出的图片尺寸一致；再将卷积结果输入到 Batch Normalization 层进行归一化后，根据可选参数输入 Dropout 层中避免过拟合，最后输入到 Leaky ReLU 激活层进行非线性变换。上采样的目标是为了放大图像补充更多细节信息。如下图所示，依次展示了、滤波器数量为 512、256、128，PCA 保留一个主成分后重建图像的效果图，可以看到随着上采样的深度增加，图像的细节保真度、锐利度更高。

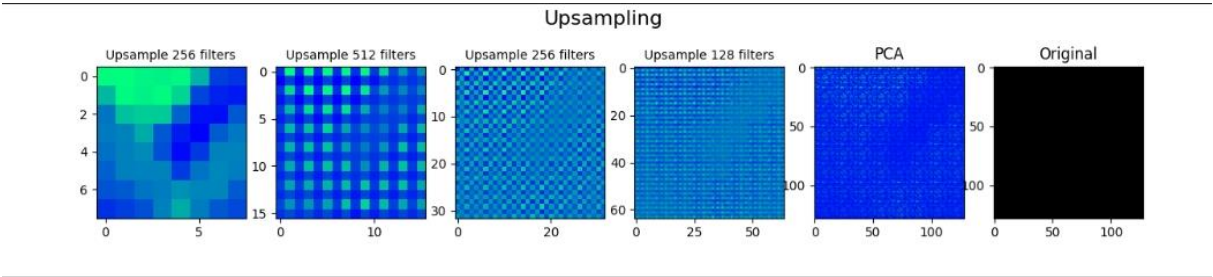


图 4.6 不同深度的上采样示意图

4.3.2 生成器结构

本文中轻量化的 CycleGAN 网络的图像输入的大小为 (128, 128) 的图片，生成器的结构如下图所示。一共由 7 个下采样模块和 7 个上采样模块组成，将输入为 (128, 128, 3) 的图像依次下采样采取特征到 (64, 64, 16)、(32, 32, 32)、(16, 16, 64)、(8, 8, 128)、(4, 4, 256)、(2, 2, 256)、(1, 1, 256) 的维度，再通过逐次上采样的操作，连接上采样和下采样维度的相同的卷积层次，最后生成大小为 (128, 128, 3) 的图片。

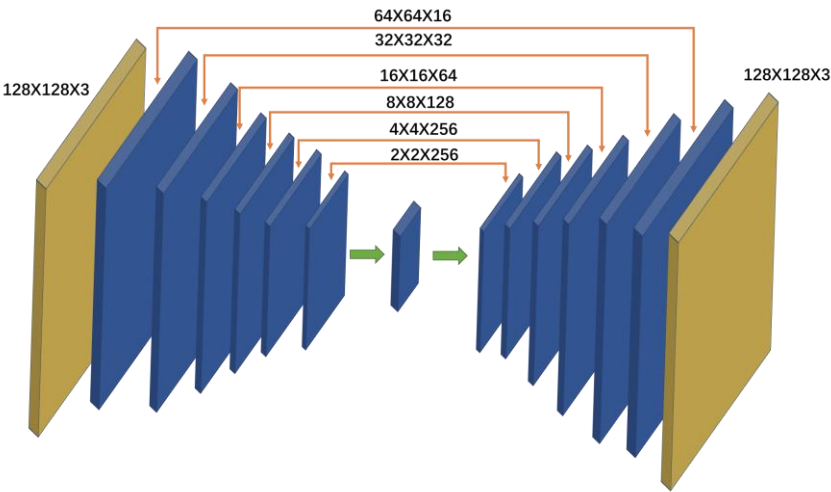


图 4.7 生成器结构示意图

生成器网络结构参数设置表如下表所示。下采样卷积和上采样转置卷积都采用 BN 归一化，除了最后的上采样层使用了“sigmoid”激活函数，其他都使用了“leaky_relu”激活函数。且在上采样的第 1、2 层中使用了 Dropout 层进行训练，防止训练过程中出现过拟合的现象。

表 4.3.2 生成器的网络结构参数表

层类型	数量	深度	归一化	激活函数	Dropout
Convolution	1	16	BN	leaky_relu	否
Convolution	1	32	BN	leaky_relu	否
Convolution	1	64	BN	leaky_relu	否
Convolution	1	128	BN	leaky_relu	否
Convolution	3	256	BN	leaky_relu	否
Deconvlution	2	256	BN	leaky_relu	是
Deconvlution	1	128	BN	leaky_relu	否
Deconvlution	1	64	BN	leaky_relu	否
Deconvlution	1	32	BN	leaky_relu	否
Deconvlution	1	16	BN	leaky_relu	否
Deconvlution	1	3	BN	sigmoid	否

输出生成器的结构信息，如下图所示，依次下采样 7 次后再进行上采样。

Layer (type)	Output Shape	Param #
Donwsample_64 (Downsample)	?	272
Donwsample_32 (Downsample)	?	2,208
Donwsample_16 (Downsample)	?	8,512
Donwsample_8 (Downsample)	?	33,408
Donwsample_4 (Downsample)	?	132,352
Donwsample_2 (Downsample)	?	263,424
Donwsample_1 (Downsample)	?	263,424
Upsample_2 (Upsample)	?	263,424
Upsample_4 (Upsample)	?	525,568
Upsample_8 (Upsample)	?	262,784
Upsample_16 (Upsample)	?	65,856
Upsample_32 (Upsample)	?	16,544
Upsample_64 (Upsample)	?	4,176
Output_layer (Upsample)	?	399

图 4.8 生成器的结构信息图

4.3.3 判别器结构

判别器的结构如下图所示。一共由 5 个下采样模块、2 个全连接层组成，将输入为 (128, 128, 3) 的图像依次下采样采取特征到 (64, 64, 32)、(32, 32, 64)、(16, 16, 128)、(8, 8, 128)、(8, 8, 256) 的维度，再逐次通过展平层转为二维后到全连接层、失活概率为 0.2 的 Dropout 层、激活函数为 “sigmoid” 的全连接层操作，最终得到一维的分类结果。

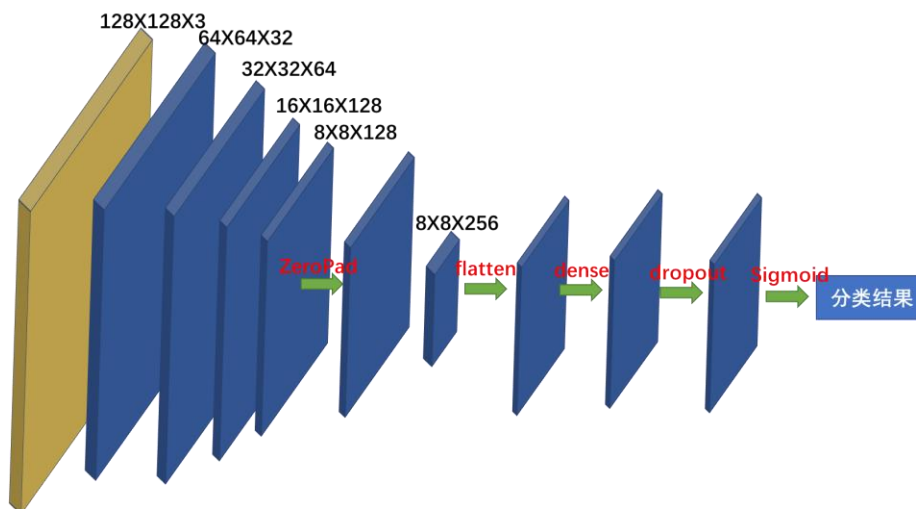


图 4.9 判别器结构示意图

输出判别器的结构信息，如下图所示。

Layer (type)	Output Shape	Param #
Downsample_64 (Downsample)	?	544
Downsample_32 (Downsample)	?	8,512
Downsample_16 (Downsample)	?	33,408
Downsample_8 (Downsample)	?	66,176
Zero_Padding (ZeroPadding2D)	?	0
Downsample_stride_1 (Downsample)	?	132,352
Flatten_layer (Flatten)	?	0
Hidden_layer_32 (Dense)	?	819,232
Dropout (Dropout)	?	0
leaky_relu (Activation)	?	0
dense (Dense)	?	33

图 4.10 判别器的结构信息图

4.4 损失计算

4.4.1 二分类交叉熵

二分类交叉熵的计算公式如下：

$$loss = -y * \log(\hat{y}) - (1 - y) * \log(1 - \hat{y})$$

其中，在二分类问题中，真实标签 y 的取值为 0 或 1，预测结果 $\hat{y}$ 的取值也为 0 或 1。

二分类交叉熵是一种常用的损失函数，它鼓励模型输出接近 1 的概率值对应正确的类别,接近 0 的概率值对应错误的类别，衡量了模型预测概率分布与真实标签分布之间的差距。

● 生成器损失

在生成器的主要目标是尽可能使生成的图片欺骗判别器分类到错误的类别，最大化判别器将生成图像判断为真实图像的概率，因此可以使用二分类交叉熵作为损失函数，使得其生成的图片更接近对应域的概率分布。生成器损失的计算公式如下：

$$L_G = -\log(D(G(z)))$$

其中， $G(z)$ 是生成的图片， $D(G(z))$ 是生成图像 $G(z)$ 为真实图像的概率。

在生成器损失中，我们将鉴别器 $D(G(z))$ 的输出与形状相同且填充有 1 的张量进行比较。该张量代表生成器的目标，创建“真实”示例。

- 判别器损失

判别器的主要目标是尽可能区分真实和生成的图片，最大化将真实图像判断为真实的概率,同时最小化将生成图像判断为真实的概率；其损失是描述判别器鉴别真实和生成图片的能力，故也可以用二分类交叉熵作为损失函数。判别器损失的计算公式如下：

$$L_G = -\log(D(x)) - \log(D(G(z)))$$

其中， $G(z)$ 是生成的图片， $D(G(z))$ 是生成图像 $G(z)$ 为真实图像的概率， $D(x)$ 为判别真实图像 x 为真实图像的概率。

在判别器损失中，使用充满 1 的张量对真实图像进行分类时比较鉴别器的输出；相反，使用充满 0 的张量对生成的图像进行分类时的鉴别器输出。

4.4.2 L1 loss

L1 loss 的计算公式如下：

$$L1\ loss = \frac{1}{N} \sum_1^N |y_i - \tilde{y}_i|$$

其中， N 是样本数量， y_i 是第 i 个样本的真实值， $\tilde{y}_i$ 是第 i 个样本的预测值。

L1 loss 也称作绝对误差损失或 Manhattan 损失,是一种常用的损失函数，计算简单高效,对异常值鲁棒性强。

- 循环一致性损失

在 Cycle GAN 网络中，循环一致性损失目标是要确保从一个域转换图像并返回的过程应生成原始图像，即 $F(G(x)) = x$ and $G(F(y)) = y$ ，生成的图像经过重建后应当与原来的图像接近。Jun-Yan Zhu*提出，将原始图像与完整循环图像进行比较并计算它们之间的平均绝对差，重建后的图像与原始域图像十分接近，这对于两个域类别都是一样的计算方法，循环一致性损失的计算公式如下：

$$L_{Cycle} = \|F(G(x)) - x\|_1 + \|G(F(y)) - y\|_1$$

根据 Jun-Yan Zhu*的工作，本文还对循环一致性损失乘于一个大小为 10 的 λ_{Cycle} 超参数，去表示循环一致性损失对于 Cycle GAN 的重要性。

- 身份损失

在 Cycle GAN 网络中，本文还添加了身份损失去衡量原始图像和身份映射图像之间的差异。身份损失有助于保留输入和输出 $G(x)$ 之间的颜色成分，但比较完整的循环图像，而是比较转换为同一域的生成器的输入和输出；身份损失的计算公式如下：

$$L_{Identity} = \|G(x) - x\|_1 + \|F(y) - y\|_1$$

同时，本文还对身份损失乘于一个大小为 5 的 $\lambda_{Identity}$ 超参数，去表示身份损失对于颜色差分重要性。

4.5 实验结果及分析

4.5.1 实验数据集

在实验中使用了来自 kaggle 网站的莫奈绘画风格和风景摄影风格数据集，如下图所示。训练集由 300 张 128x128 尺寸的莫奈绘画风格图像和 7000 张 128x128 风景摄影风格图像，测试集合由 15 张 128x128 尺寸的莫奈绘画风格图像和 15 张 128x128 风景摄影风格图像。



图 4.11 莫奈风格和风景摄影风格数据集

4.5.2 实验对比及分析

(1) 无感知损失

- 在固定学习率为 $2e-4$ ，训练批次大小为 64，训练轮次为 30 次下；训练每隔 5 次使用轻量化的 Cycle GAN 的在验证集上进行验证，训练效果如下图所示。

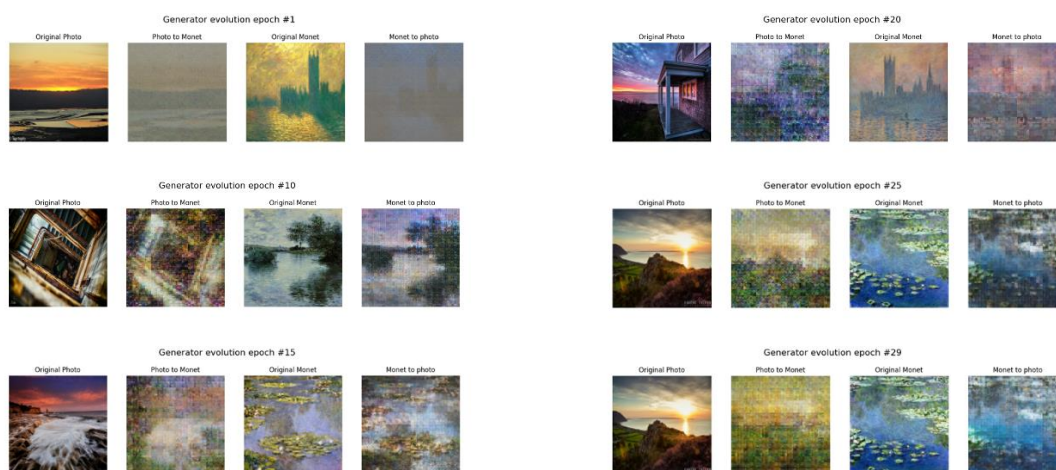


图 4.12 训练效果示意图

生成器、判别器对应的损失函数如下图所示。

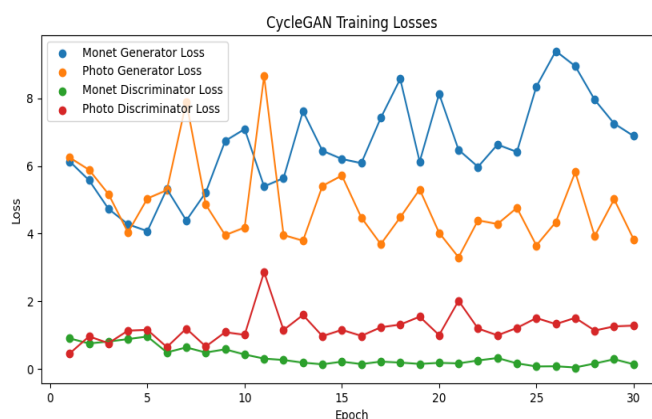


图 4.13 损失函数曲线图

轻量化的 Cycle GAN 模型在测试集上的转换效果如下图所示。

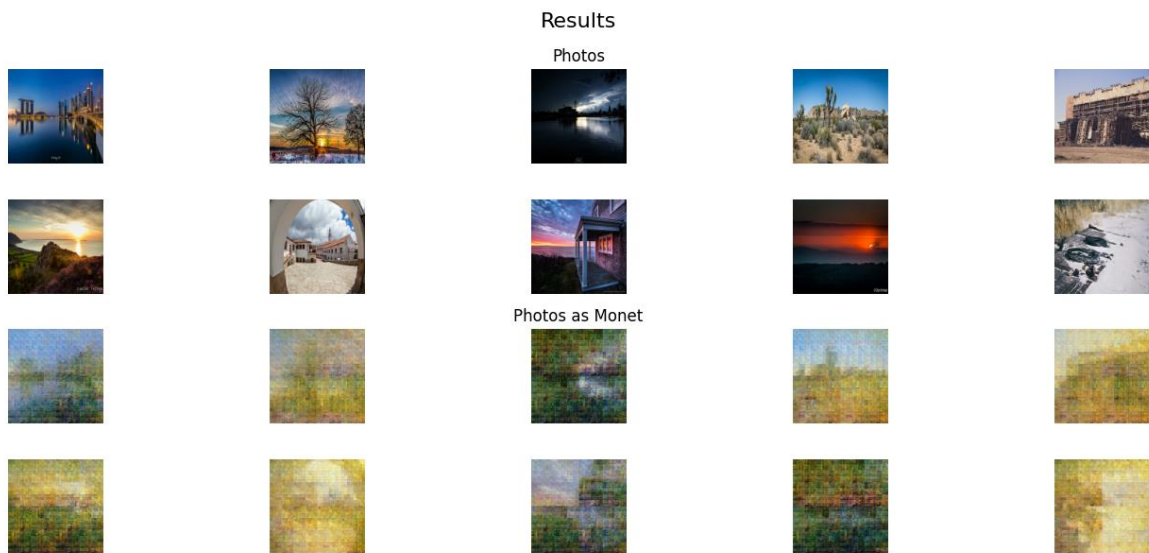


图 4.14 测试集转换效果图

- 继续保持学习率为 $2e-4$ ，训练批次大小为 64，将训练轮次改为 100 次；训练每隔 20 次使用轻量化的 Cycle GAN 的在验证集上进行验证，训练效果如下图所示。

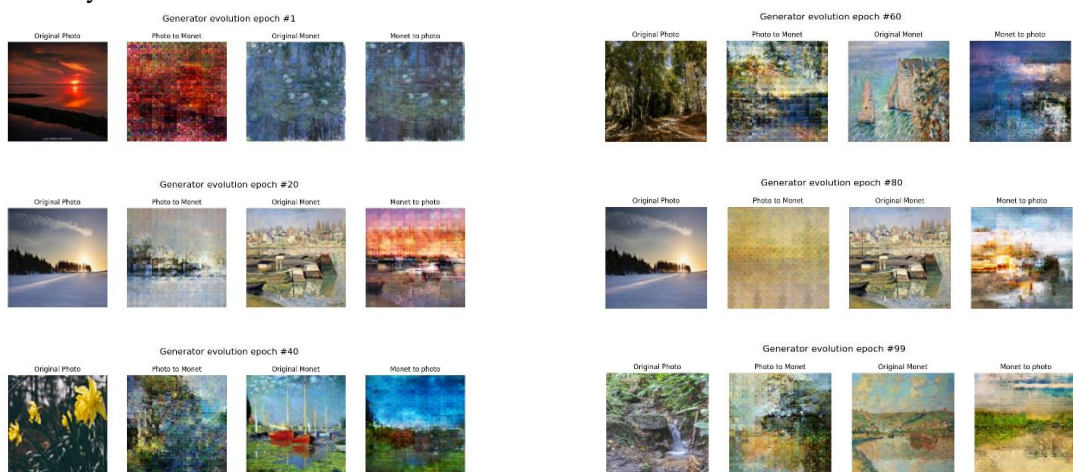


图 4.15 训练效果示意图

生成器、判别器对应的损失函数如下图所示。

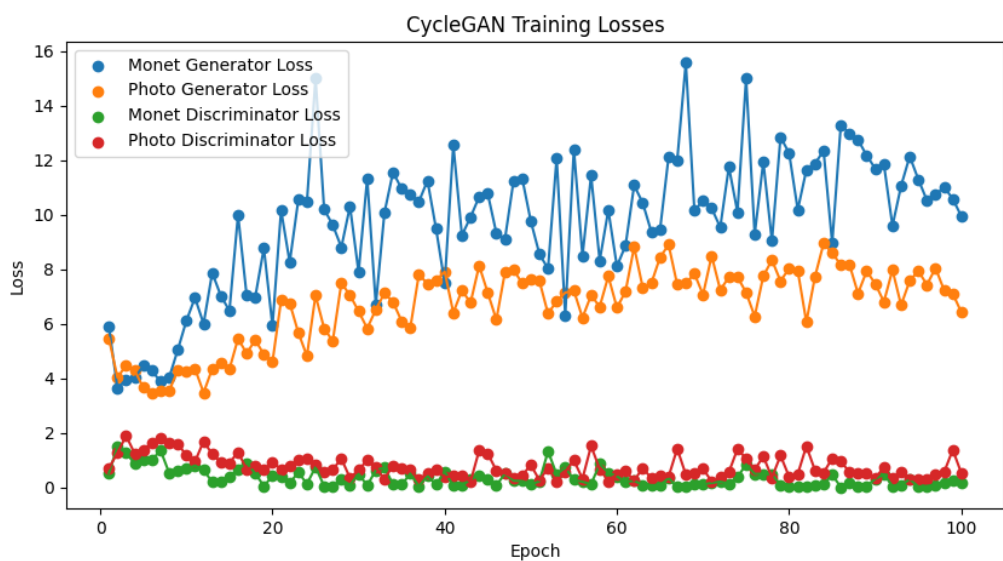


图 4.16 损失函数曲线图

轻量化的 Cycle GAN 模型在测试集上的转换效果如下图所示。

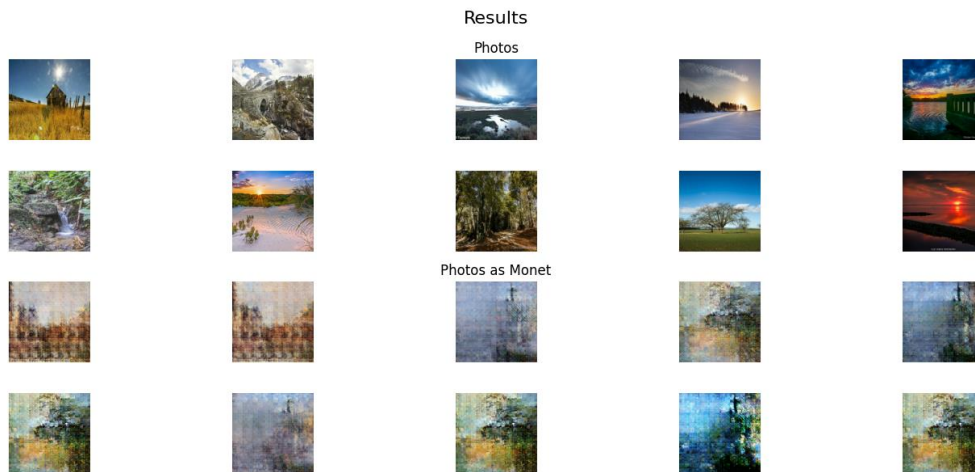


图 4.17 测试集转换效果图

- 在固定学习率为 $2e-3$ ，训练批次大小为 64，训练轮次为 30 次下；训练每隔 5 次使用轻量化的 Cycle GAN 的在验证集上进行验证，训练效果如下图所示。

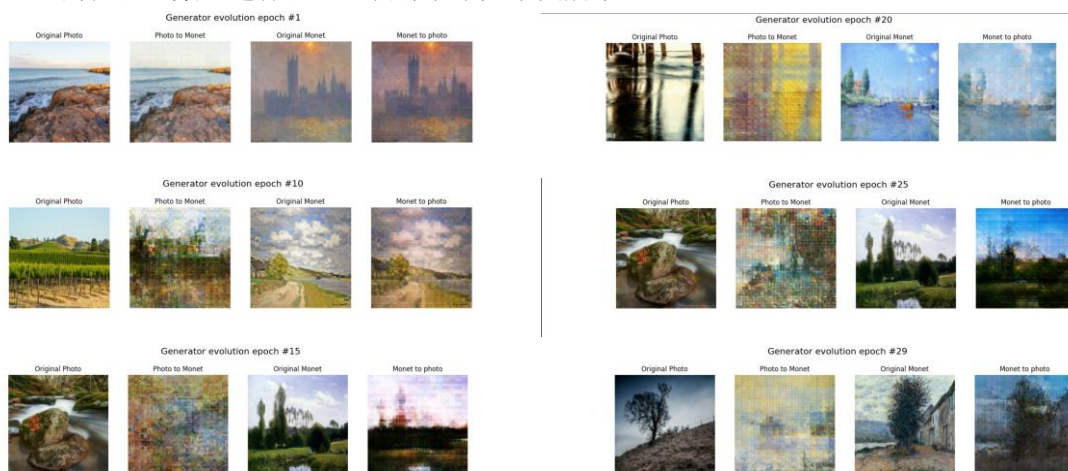


图 4.18 训练效果示意图

生成器、判别器对应的损失函数如下图所示。

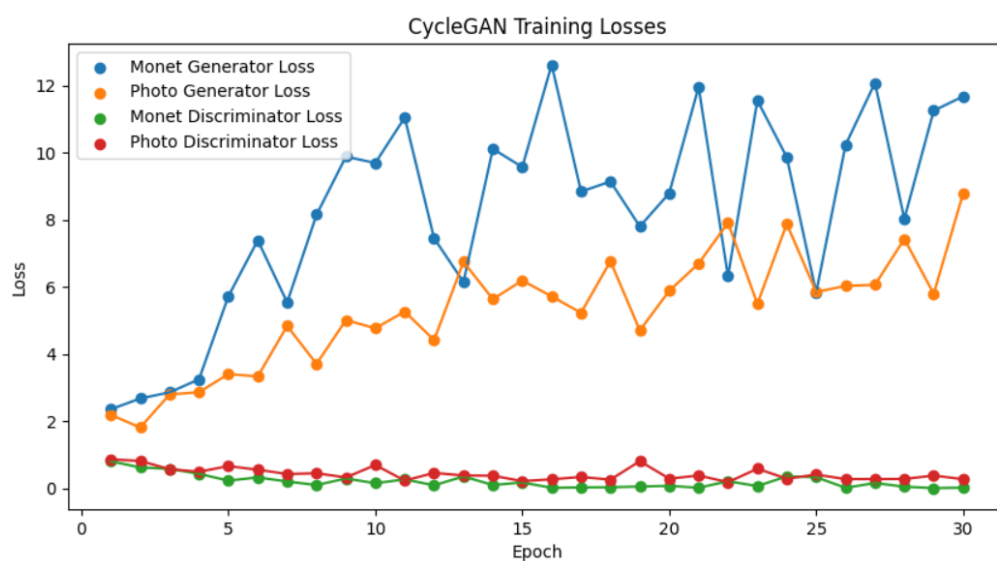


图 4.19 损失曲线图

轻量化的 Cycle GAN 模型在测试集上的转换效果如下图所示。

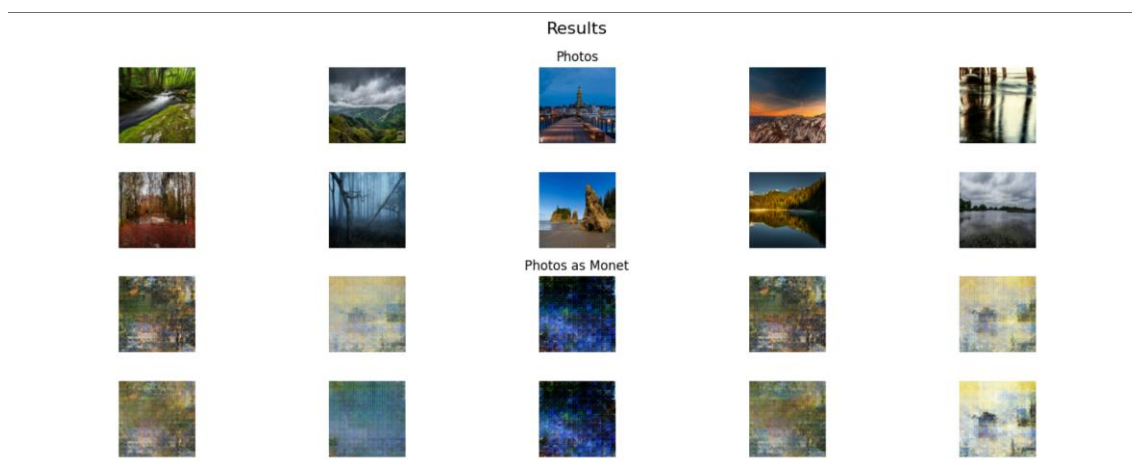


图 4.20 测试集转换效果图

- 由上述实验可以看到，没有加入感知损失的轻量化的 Cycle GAN 在迭代 10-20 轮时几乎收敛，而且在学习率为 $2e-3$ 时模型收敛速度更快，因此可以通过微调学习率进行重新训练：模型前 20 轮的学习率为 $2e-3$ ，后续的学习率为 $2e-4$ 。模型训练效果如下图所示。

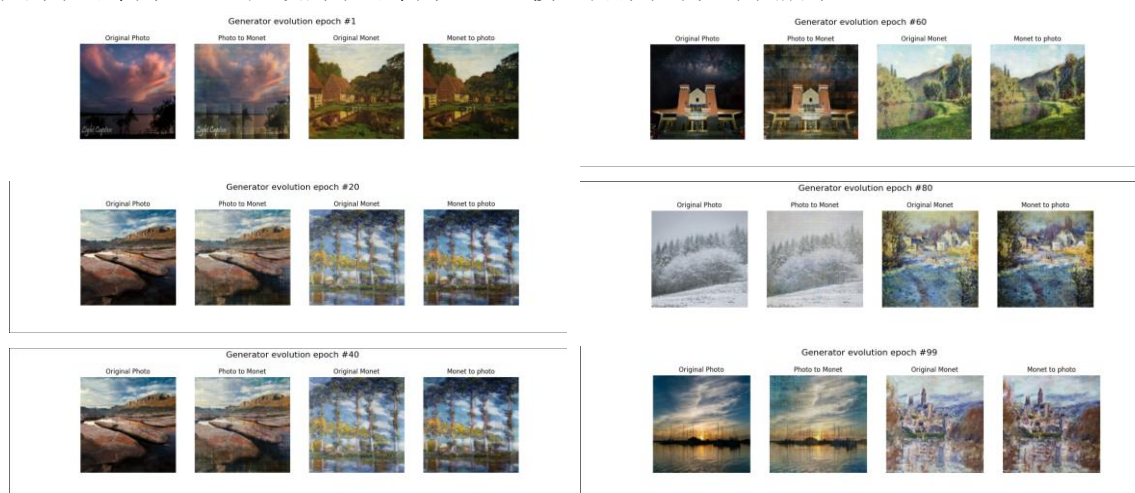


图 4.21 训练效果示意图

生成器、判别器的损失曲线图如下图所示。

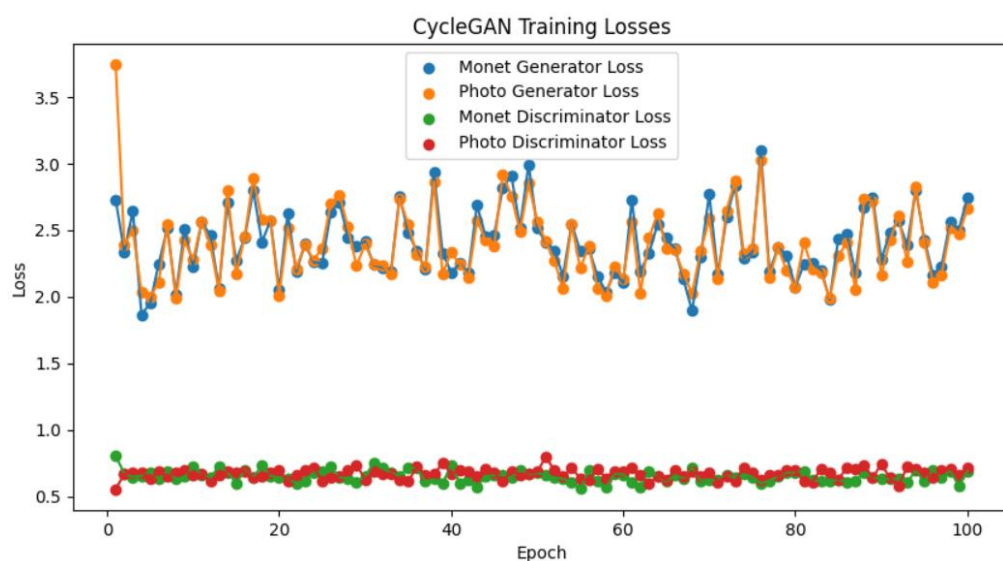


图 4.22 损失曲线图

轻量化的 Cycle GAN 模型的最终转换效果，如下图所示。

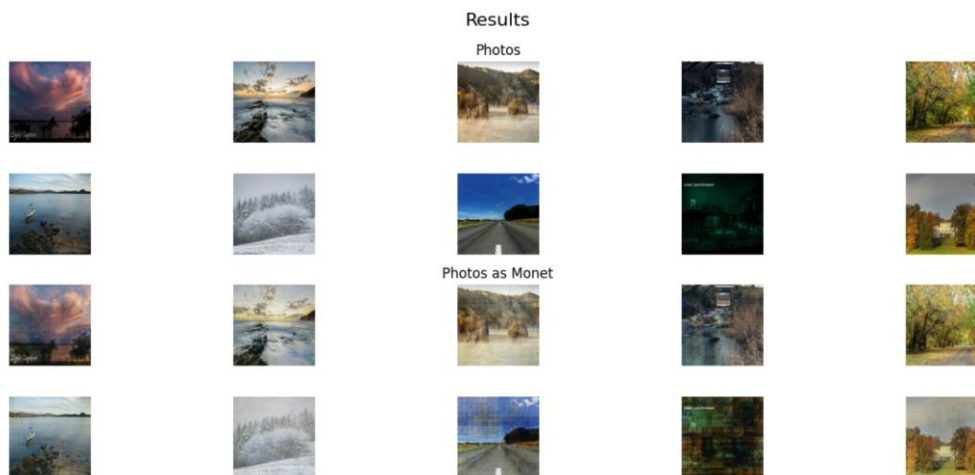


图 4.23 测试集转换效果图

● 结论

从起始的固定学习率为 $2e-4$ 训练 30、100 次再到固定学习率为 $2e-3$ 训练 30 次，可以发现模型的收敛速度更快，经过最后的学习率微调：前约 20 轮的学习率为 $2e-3$ ，后续的学习率为 $2e-4$ ；模型最终拟合的效果更好，训练的更稳定，两个生成器的损失变化更加稳定，波动范围更小；因此，可以认为，该轻量化的 CycleGAN 模型已经收敛。不仅如此，可以发现微调后的模型在图像转换的稳定性更优，更加逼真地呈现了真实风景图像向莫奈风格的转换。

(2) 加入感知损失

● 在固定学习率为 $2e-4$ ，训练批次大小为 64，训练轮次为 30 次下；训练每隔 5 次使用轻量化的 CycleGAN 的在验证集上进行验证，训练效果如下图所示。

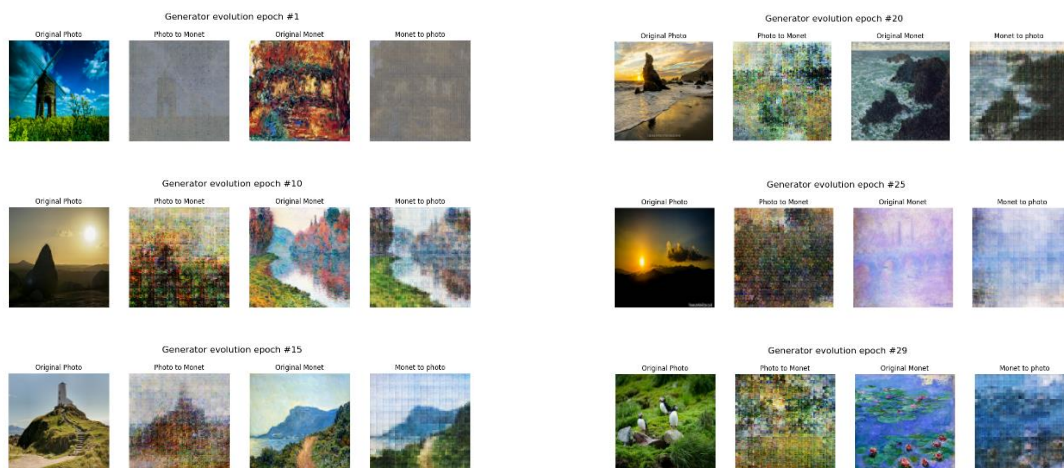


图 4.24 训练效果示意图

生成器、判别器对应的损失函数如下图所示。

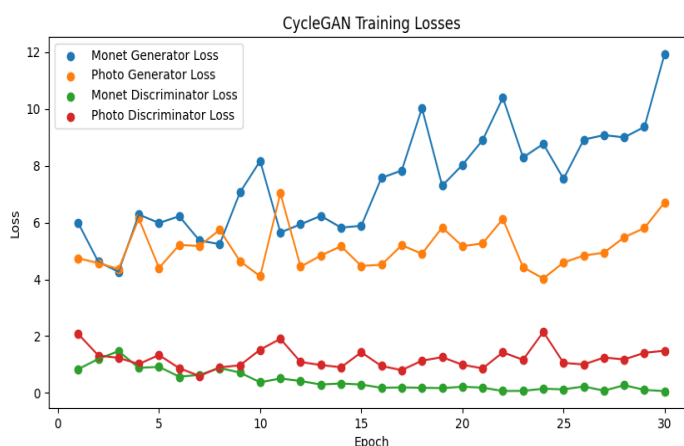


图 4.25 损失函数曲线图

轻量化的 Cycle GAN 模型在测试集上的转换效果如下图所示。

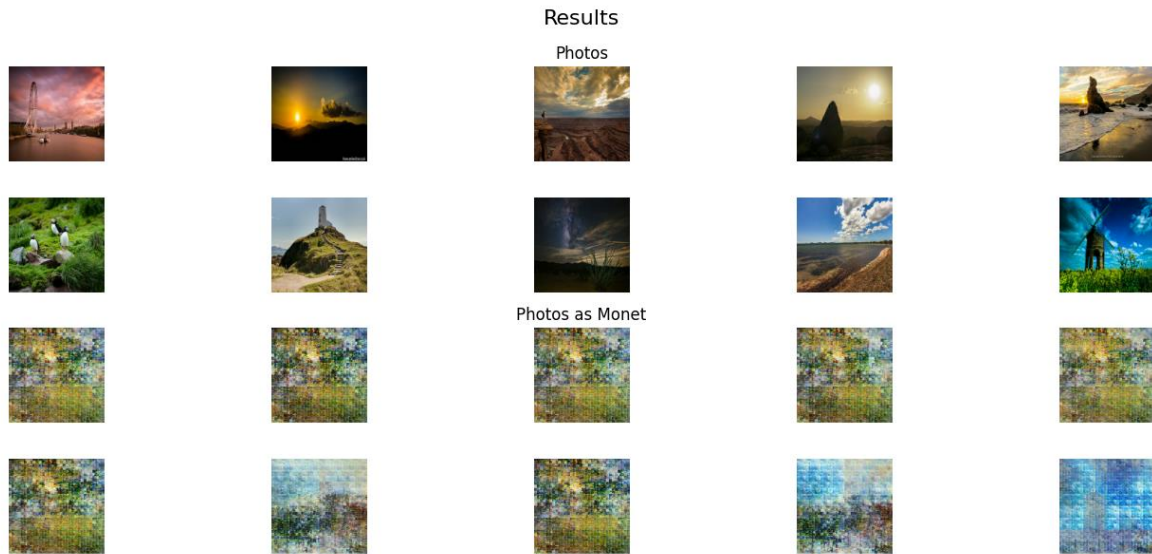


图 4.26 测试集转换效果图

- 继续保持学习率为 $2e-4$ ，训练批次大小为 64，将训练轮次改为 100 次；训练每隔 20 次使用轻量化的 Cycle GAN 的在验证集上进行验证，训练效果如下图所示。



图 4.27 训练效果示意图

生成器、判别器对应的损失函数如下图所示。

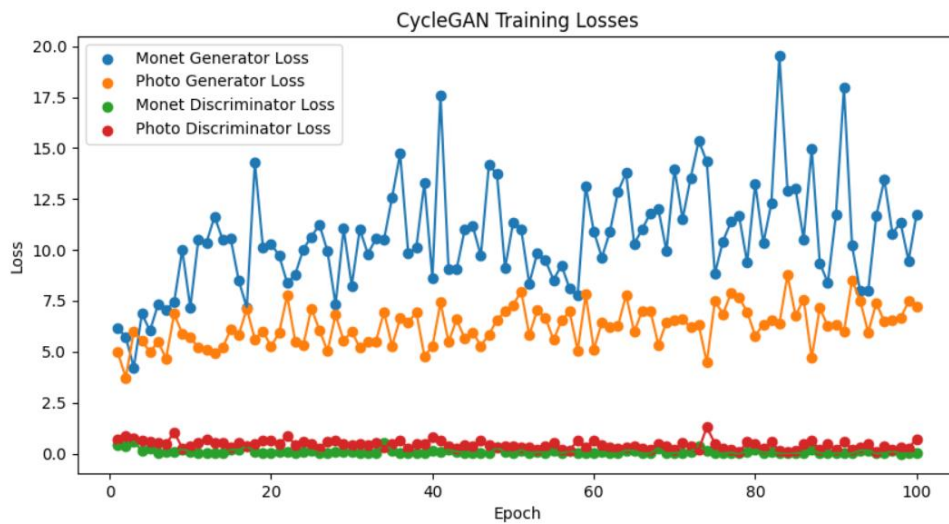


图 4.28 损失函数曲线图

- 在提高固定学习率为 $2e-3$ ，训练批次大小为 64，训练轮次为 30 次下；训练每隔 5 次使用轻量化的 Cycle GAN 的在验证集上进行验证，训练效果如下图所示。

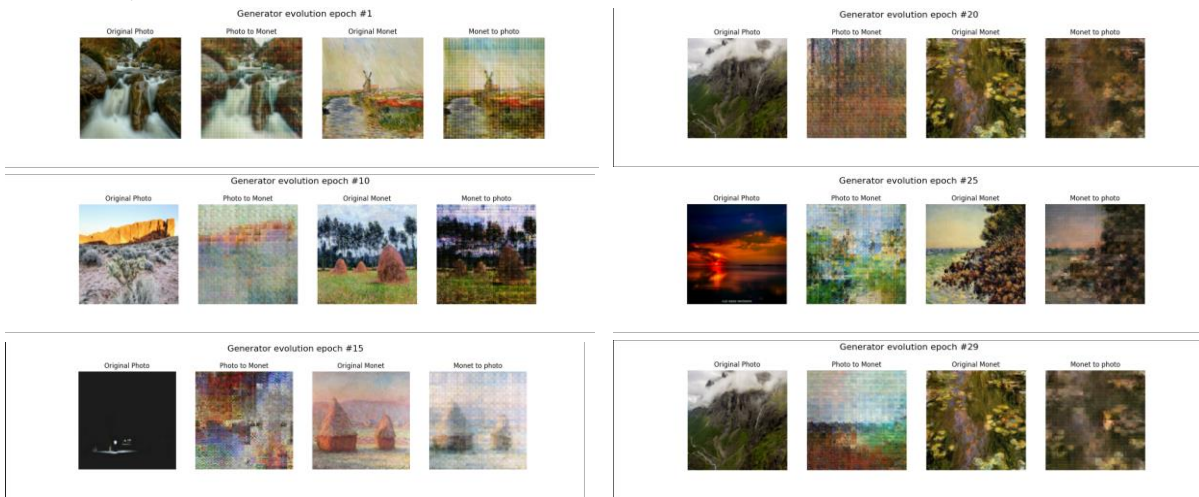


图 4.29 训练效果示意图

生成器、判别器的损失曲线图如下图所示。

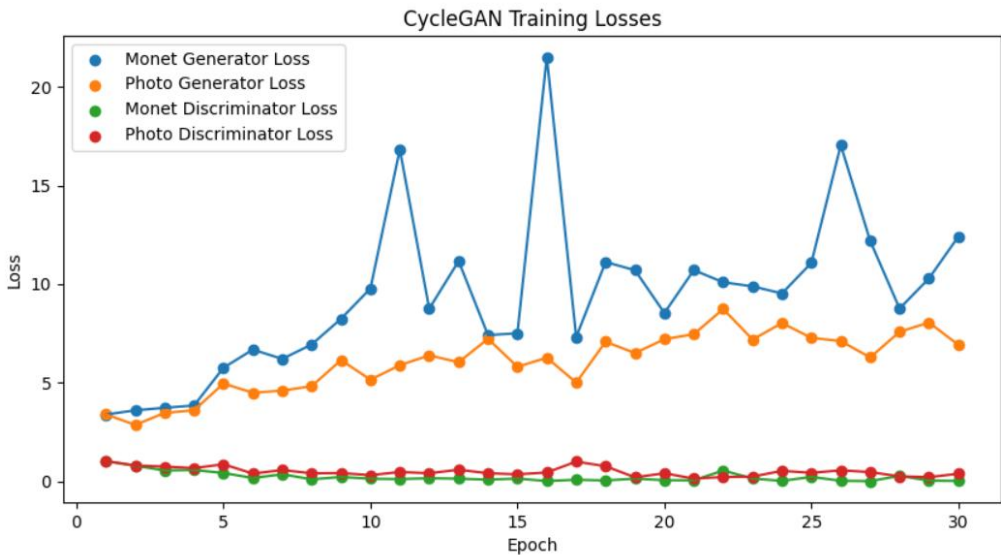


图 4.30 损失曲线示意图

轻量化的 Cycle GAN 模型在测试集上的转换效果如下图所示。

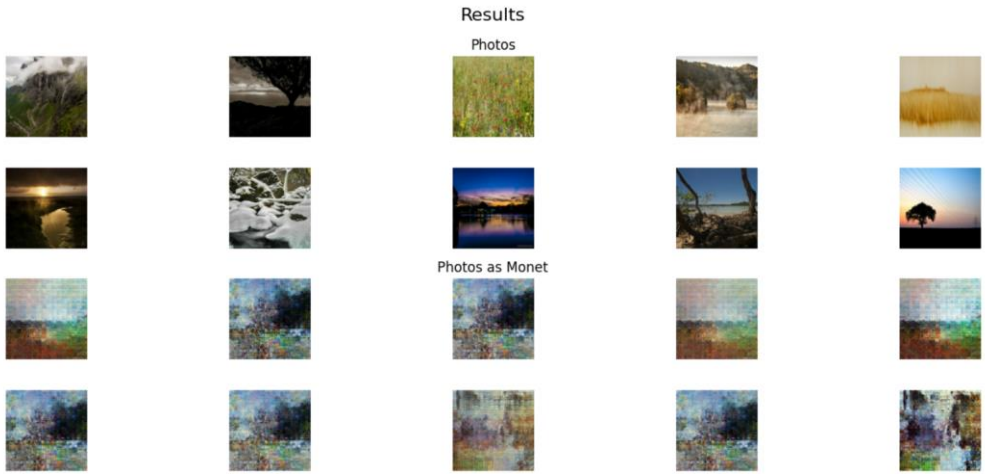


图 4.31 测试集转换效果图

- 由上述实验可以看到，加入感知损失的轻量化的 Cycle GAN 同样是在迭代 10-20 轮时几乎收敛，而且在学习率为 $2e-3$ 时模型收敛速度更快，因此，我们同样通过微调学习率进行重新训练：模

型前 20 轮的学习率为 $2e-3$ ，后续的学习率为 $2e-4$ 。模型训练效果如下图所示。

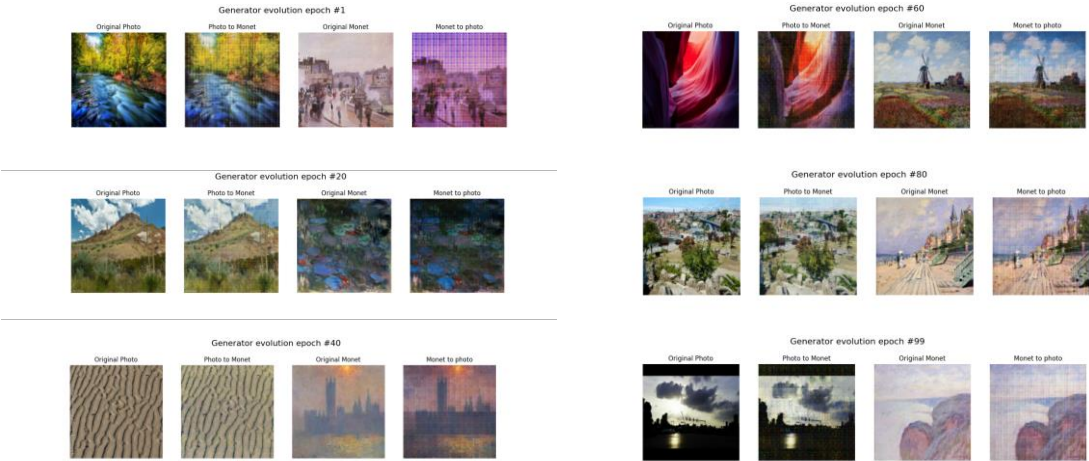


图 4.32 训练效果示意图

生成器、判别器的损失曲线图如下图所示。

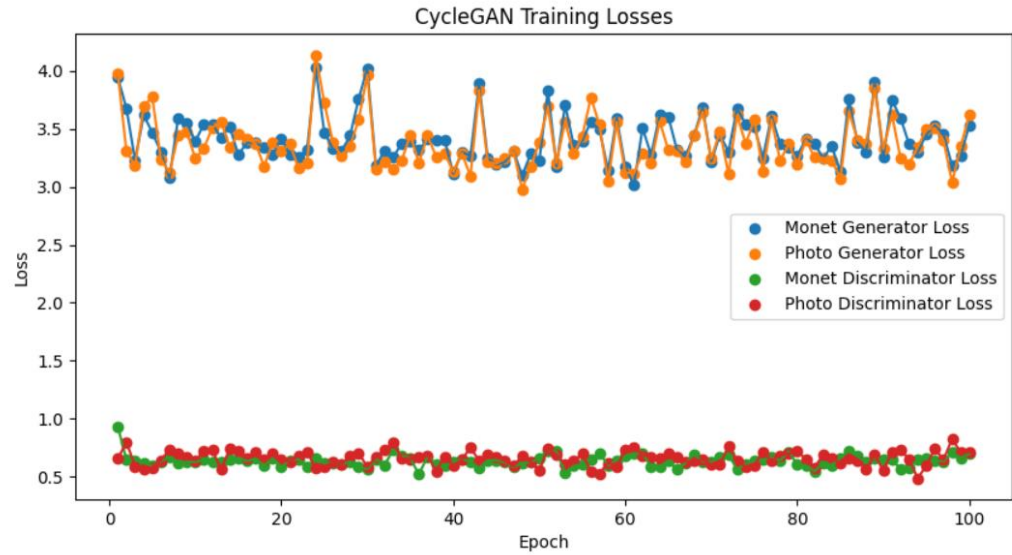


图 4.33 损失曲线图

轻量化的 Cycle GAN 模型的最终转换效果，如下图所示。

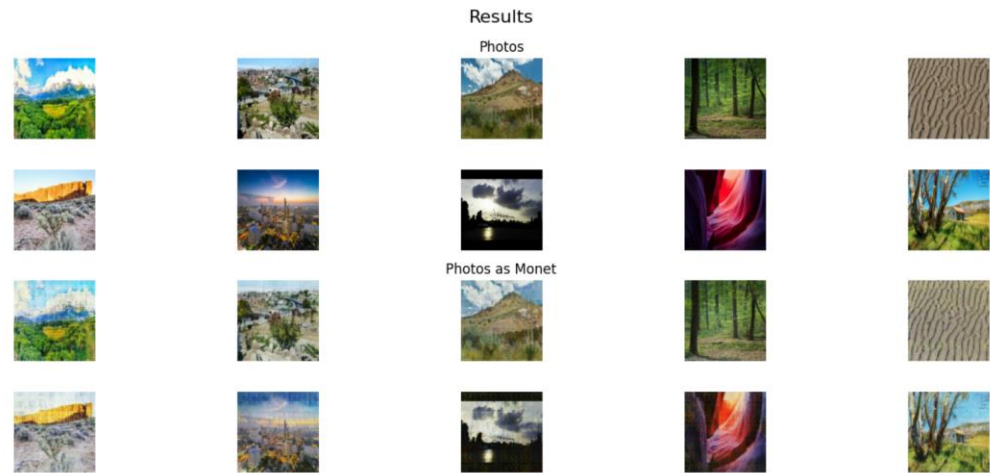


图 4.34 测试集转换效果图

● 结论

从起始的固定学习率为 $2e-4$ 训练 30、100 次再到固定学习率为 $2e-3$ 训练 30 次，可以发现该模型的收敛速度更快，经过最后的学习率微调：前约 20 轮的学习率为 $2e-3$ ，后续的学习率为 $2e-4$ ；模

型最终拟合的效果更好，两个生成器的损失变化更加稳定，波动范围也更小。

因此，可以认为，该加入感知损失的轻量化的 CycleGAN 模型已经收敛。不仅如此，可以发现微调后的模型在图像转换的稳定性更优，判别器的损失较没加入感知损失的模型的更加稳定，即在一定的小范围内波动；生成器的损失也较没加入感知损失的模型更加稳定、对应的损失更小；更加逼真地呈现了真实风景图像向莫奈风格的转换，图片的色调较没有感知损失的更加自然、更加贴近真实风格图片。

(3) 对比分析

对没有加入、加入感知损失的 CycleGAN 模型进行效果分析，以下依次是两个模型的微调学习率后的损失曲线图。可以发现加入感知损失后的 CycleGAN 的生成器损失变化比没有感知损失的更加稳定，二者的判别器损失几乎一致；但加入感知损失后的 CycleGAN 的生成器损失比没有加入感知损失的高（因为感知损失就是加给生成器），属于正常现象。

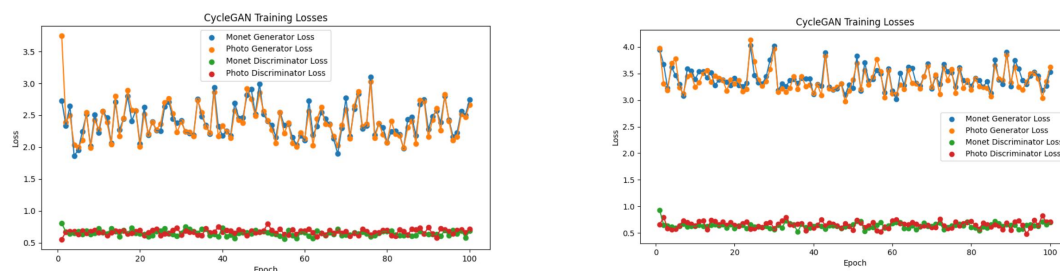


图 4.34 损失曲线对比图（左为无感知损失，右为有感知损失）

没有加入、加入感知损失的 CycleGAN 模型在测试集的准换效果如下图所示。可以发现加入感知损失的图片效果比没有感知损失的更好，同时也保留了更多关于莫奈风格的细节，使得图片转换的色调、风格过渡的更加自然。

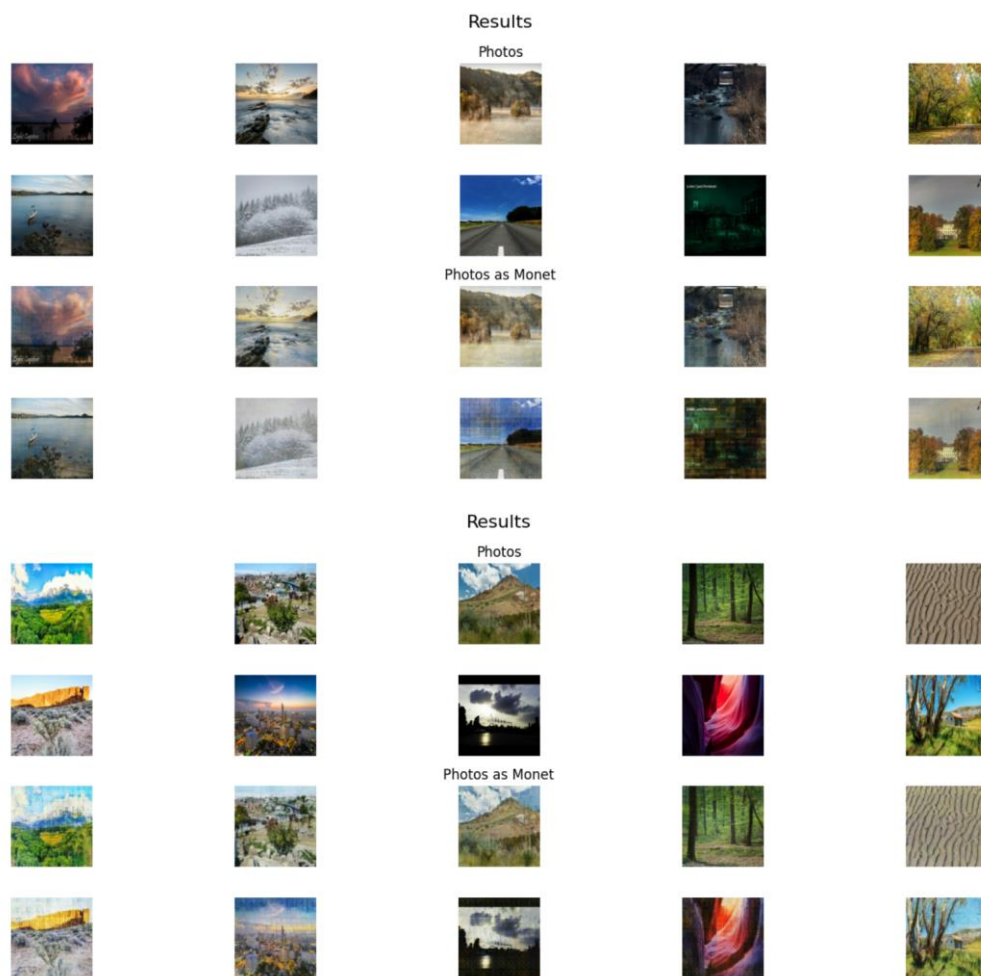


图 4.35 测试集转换效果图（上为无感知损失，下为加入感知损失）

5 Cycle GAN 模型

本章节主要介绍第二种 Cycle GAN 模型，输入图像的尺寸从 (128, 128) 变为 (256, 256)，在生成器部分增加了 Residual Block 模块，提取图像的更多特征，解决反向传播中梯度消失或爆炸的问题，使生成器生成的图片更具有欺骗性（欺骗判别器）。同样，也在生成器中添加了轻量化的预训练 MobileNetV2 模型进行感知损失的计算，使得生成的图片细节更丰富、风格更自然。

5.1 总体模型

该 Cycle GAN 模型同样是由生成器和判别器构成。与轻量化的 Cycle GAN 模型的区别就是模型输入的尺寸大小为 (256, 256)、模型中添加了 Residual Block 残差块。在模型训练学习过程中,将生成器生成的图片依次输入到残差网络和 MobileNetV2 中计算感知损失后联合输出图片，缓解梯度消失/爆炸问题的同时可降低模型学习时的误差而且使得生成的图像细节和质感更加自然。在 Loss 中添加了感知损失函数，使用轻量化的 MobileNetV2 模型进行计算，在保持内容结构的同时，生成的图像接近于风格化的图像。风格转换模型如下图 5.1 所示。

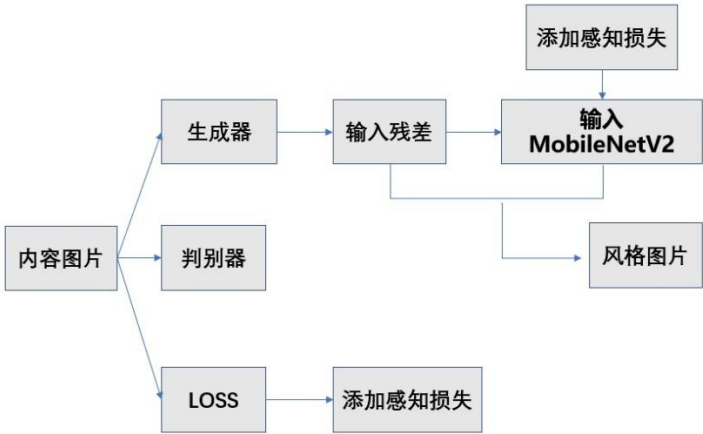


图 5.1 风格转换模型

5.2 残差块

残差块是深度神经网络模型中的一种关键组件,最初被用于 ResNet(Residual Network)中。残差块的核心思想是,通过在网络层之间添加"跳跃连接"(shortcut connection),可以让网络更容易学习到恒等映射(identity mapping)。这种残差连接可以让梯度在反向传播过程中更容易流动,缓解了深度网络中常见的梯度消失或爆炸问题。

如下图 5.2 所示，残差块通常由两个或三个串联的卷积层组成,并且在这些层之间添加了一条跳跃连接。跳跃连接将输入直接添加到最终输出上使得网络可以更容易学习恒等映射,从而提高整体性能

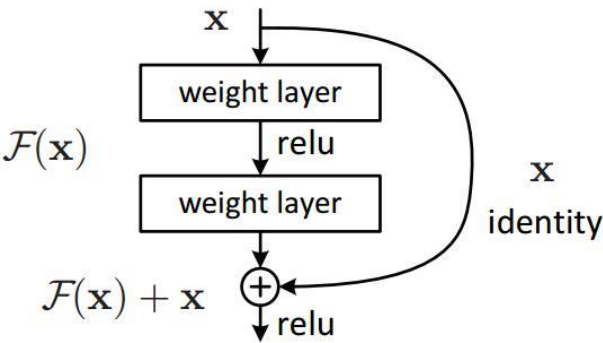


图 5.2 残差网络原理示意图

5.3 模型实现

该 CycleGAN 模型下采样和上采样模块不变，输入的图像大小从轻量化的（128，128）变为了（256，256），本节将逐步介绍 Residual Block 块、生成器结构、判别器结构。

5.3.1 Residual Block

本文中设计的残差块由两个深度为 512、卷积核大小为（3，3）、步长为 1 的 Conv2D 二维卷积层，两个 Batch Normalization 层，1 个 Leaky ReLU 激活层组成。残差块部分先通过第一个 Conv2D 层和 Batch Normalization 层后通过 Leaky ReLU 激活层，再经过第二个 Conv2D 层和 Batch Normalization 层后输出。Residual Block 模块是为了缓解梯度消失/爆炸，提高网络深度，增强网络的学习能力。Residual Block 模块的结构如下图所示。

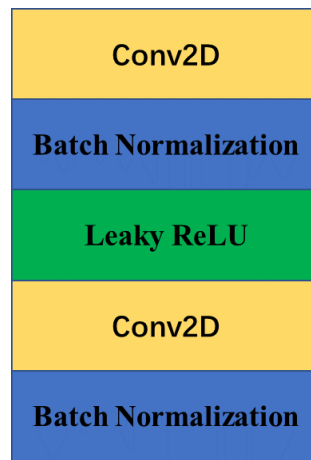


图 5.3 Residual Block 模块示意图

5.3.2 生成器结构

本文中轻量化的 CycleGAN 网络的图像输入的大小为（256，256）的图片，生成器的结构如下图所示。一共由 7 个下采样模块、3 个残差块和 7 个上采样模块组成，将输入为（256，256，3）的图像依次下采样采取特征到（128，128，128）、（64，64，128）、（32，32，256）、（16，16，512）、（8，8，512）、（4，4，512）的维度后，先通过 3 次残差块，再通过逐次上采样的操作，连接上采样和下采样维度的相同的卷积层次，最后生成大小为（256，256，3）的图片。

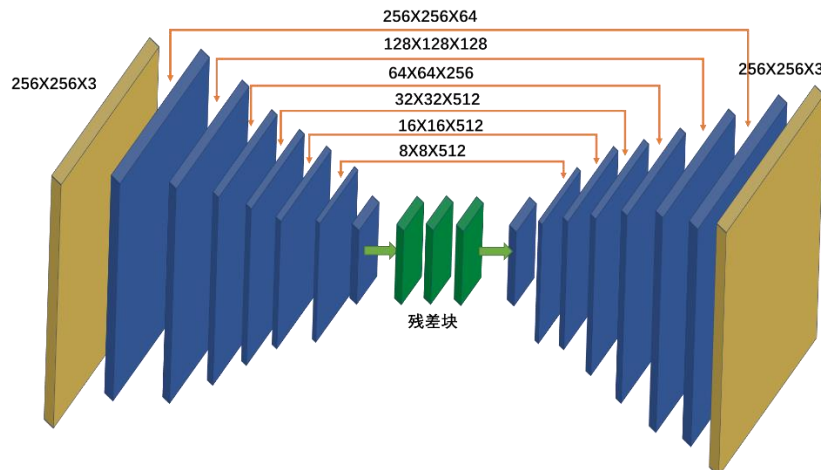


图 5.4 生成器结构示意图

生成器网络结构参数设置表如下表所示。下采样卷积、采样转置卷积和残差块都采用 BN 归一

化，除了最后的上采样层使用了“tanh”激活函数，其他都使用了“leaky_relu”激活函数。

表 5.3.2 生成器的网络结构参数表

层类型	数量	深度	归一化	激活函数
Convolution	1	64	BN	leaky_relu
Convolution	2	128	BN	leaky_relu
Convolution	1	256	BN	leaky_relu
Convolution	3	512	BN	leaky_relu
Redisual	3	512	BN	leaky_relu
Deconvlution	2	512	BN	leaky_relu
Deconvlution	1	256	BN	leaky_relu
Deconvlution	2	128	BN	leaky_relu
Deconvlution	1	64	BN	leaky_relu
Deconvlution	1	3	BN	sigmoid

5.3.3 判别器结构

判别器的结构如下图所示。一共由 5 个下采样模块、2 个全连接层组成，将输入为（256，256，3）的图像依次下采样采取特征到（128，128，64）、（64，64，128）、（32，32，256）、（32，32，512）的维度，再逐次通过展平层转为二维后到全连接层、失活概率为 0.2 的 Dropout 层、激活函数为“sigmoid”的全连接层操作，最终得到一维的分类结果。

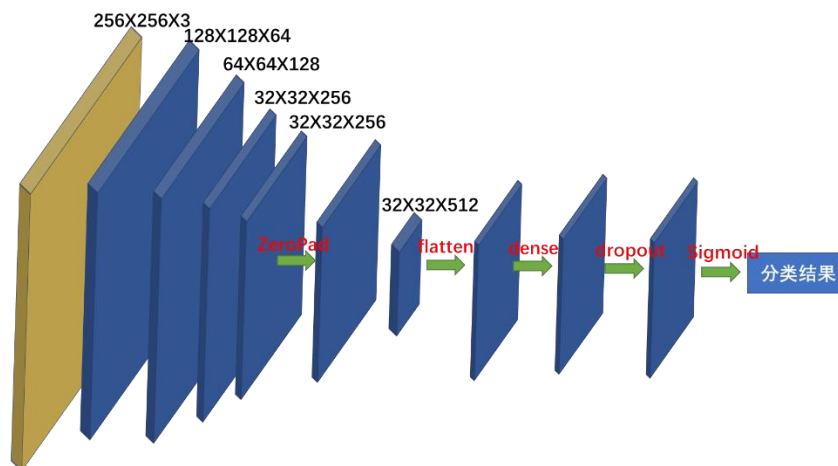


图 5.5 判别器结构示意图

5.4 实验结果及分析

5.4.1 实验数据集

在实验中使用了来自 kaggle 网站的莫奈绘画风格和风景摄影风格数据集，如下图所示。训练集由 300 张 256x256 尺寸的莫奈绘画风格图像和 7000 张 256x256 风景摄影风格图像，测试集合由 15 张 256x256 尺寸的莫奈绘画风格图像和 15 张 256x256 风景摄影风格图像。



图 5.6 莫奈风格和风景摄影风格数据集

5.4.2 实验结果

在固定学习率为 $2e-4$ ，为选择器和判别器使用”adam”优化器作为反向传播时的参数传播，训练批次大小为 15。

生成器的结构可视化如下图所示。

Layer (type)	Output Shape	Param #
Donwsample_same (Downsample)	?	5,120
Donwsample_128 (Downsample)	?	33,408
Donwsample_64 (Downsample)	?	66,176
Donwsample_32 (Downsample)	?	132,352
Donwsample_16 (Downsample)	?	526,848
Donwsample_8 (Downsample)	?	1,051,136
Donwsample_4 (Downsample)	?	1,051,136
residual_block (ResidualBlock)	?	4,723,712
residual_block_1 (ResidualBlock)	?	4,723,712
residual_block_2 (ResidualBlock)	?	4,723,712
Upsample_8 (Upsample)	?	1,051,136
Upsample_16 (Upsample)	?	2,099,712
Upsample_32 (Upsample)	?	1,049,856
Upsample_64 (Upsample)	?	262,784
Upsample_128 (Upsample)	?	131,712
Upsample_256 (Upsample)	?	65,856
Output_layer (Upsample)	?	1,551

图 5.7 生成器结构示意图

对生成器的初始生成效果进行可视化，结果如下图所示。可以看到，生成器提取了许多图像的特征。

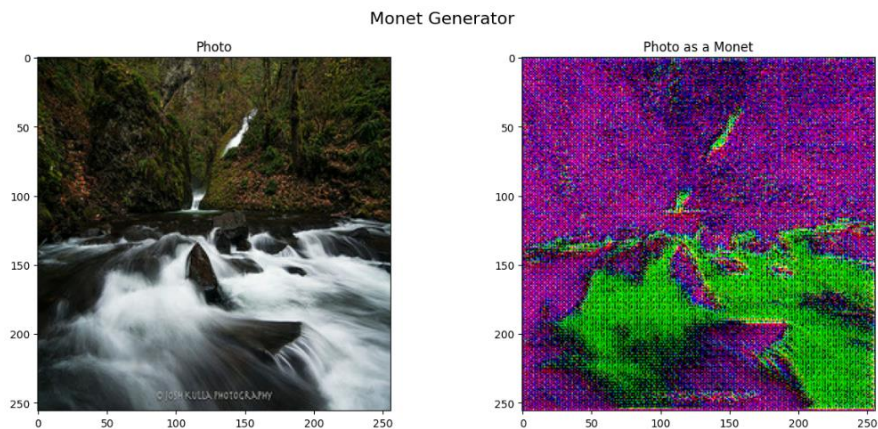


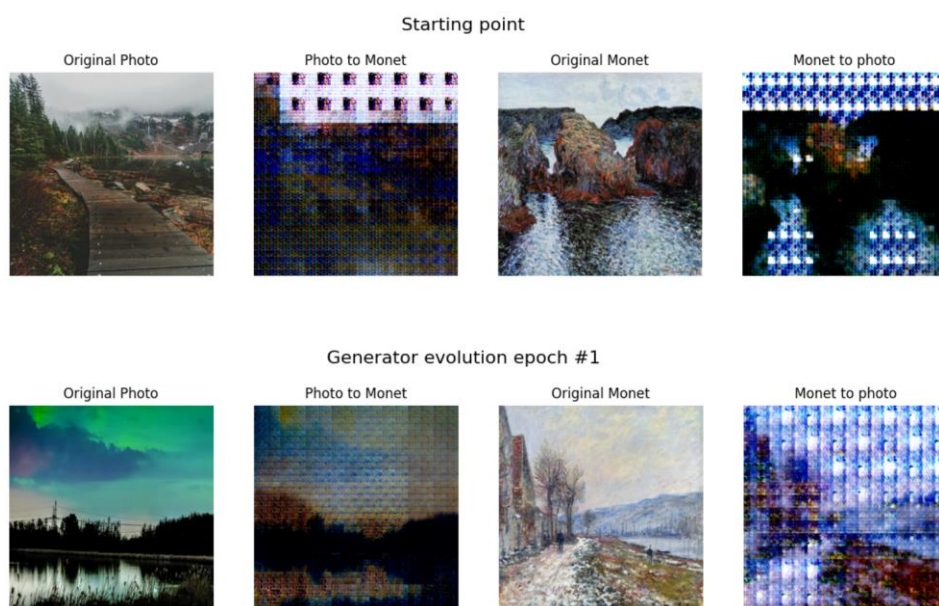
图 5.8 生成器可视化效果图

判别器的结构可视化如下图所示。

Layer (type)	Output Shape	Param #
Downsample_128 (Downsample)	?	1,088
Downsample_64 (Downsample)	?	33,408
Downsample_32 (Downsample)	?	132,352
Downsample_16 (Downsample)	?	263,424
Zero_Padding (ZeroPadding2D)	?	0
Downsample_stride_1 (Downsample)	?	526,848
Flatten_layer (Flatten)	?	0
Hidden_layer_64 (Dense)	?	10,616,896
Dropout (Dropout)	?	0
leaky_relu (Activation)	?	0
dense (Dense)	?	65

图 5.9 判别器结构示意图

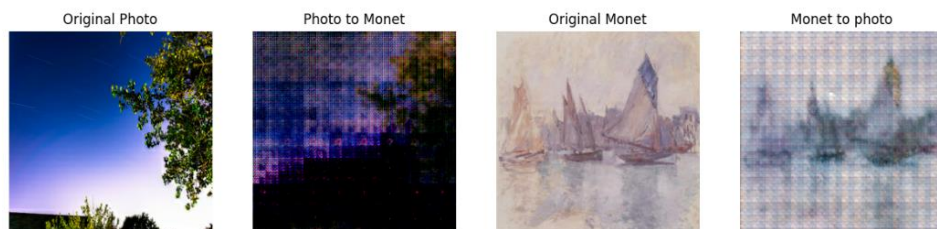
由于模型训练时间过长且 GPU 的内存大小不足以支持训练，只能训练 7 轮，训练效果如下图所示。



Generator evolution epoch #2



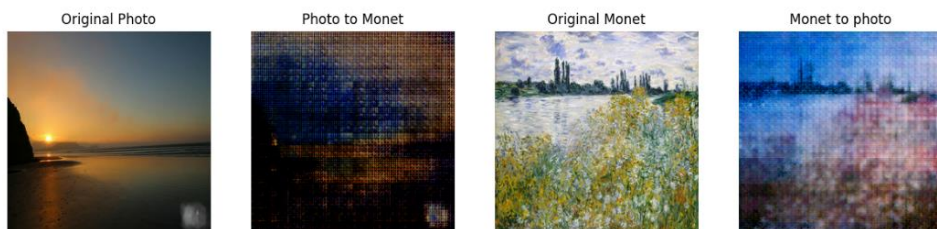
Generator evolution epoch #3



Generator evolution epoch #4



Generator evolution epoch #5



Generator evolution epoch #6



Generator evolution epoch #7



图 5.10 训练效果示意图

最终的转换效果如下图所示

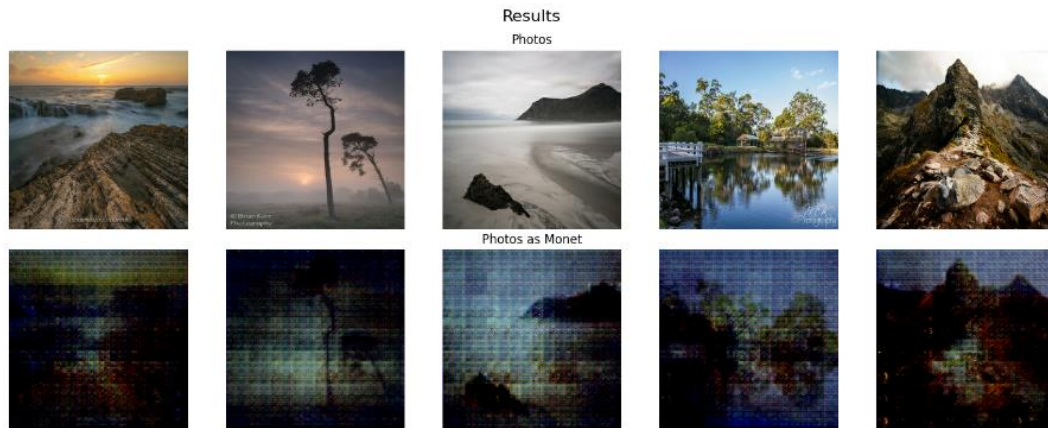


图 5.11 转换效果图

● 结论

由如图 5.7 所示的训练效果可以看到，该 Cycle GAN 模型很好地实现了对真实风景图片向莫奈风格图片的转换，学习了更多的莫奈特征细节，真实风景转换后的图片极大地显现了原图片的细节及特征；且对莫奈风格特征的学习能力比轻量化模型更好、更加细致；该模型的学习能力也更强。加入残差块后的 Cycle GAN 模型比没有残差块的模型在图片特征学习、图片风格迁移的效果更强。

6 总结与展望

本文主要研究了基于 Cycle GAN 的莫奈风格图片转换，根据输入图像的尺寸大小以及模型结构不同，设计了两种 Cycle GAN 模型：轻量化的 Cycle GAN 模型、加入残差块的 Cycle GAN 模型。在 Cycle GAN 的构建中，均从 Cycle GAN 的结构出发，从 Cycle GAN 的构成原理分析，逐步地实现 Cycle GAN 的生成器、判别器模块。实验结果表明，以上两种 Cycle GAN 模型在训练次数充足的情况下拥有很好的莫奈风格转换效果。本文的主要研究成果如下：

1. 详细分析了通过下采样和上采样操作进行特征提取学习去构建生成器、判别器的过程及原理，设计了针对输入尺寸为 (128, 128)、且收敛速度更快的轻量化的 Cycle GAN 模型；得出了在莫奈风格转换的工作中使用 MobileNetV2 模型计算感知损失可以提高模型训练的稳定性结论，使得转换后的图片风格更加自然，转换效果更好。
2. 详细介绍了 Residual Block 残差块的作用，在生成器模块中添加了 3 层残差块增强生成器的学习能力，并将生成器、判别器的输入大小改为 (256, 256)，使得加入残差块的 Cycle GAN 模型能够学习更多的特征；结果表明该加入残差块的 Cycle GAN 模型很好的完成了莫奈风格图片转换工作。

同时，本文的研究依旧存在许多的不足和局限性：

1. 微调后的轻量化的 Cycle GAN 模型的学习能力仍然有限，在初始下采样时采集的图像特征不够充足，模型在训练次数超过 50 次后，学习能力达到瓶颈；即使生成器、判别器的损失稳定且模型收敛，但转换后的图片更加偏向于真实风格。
2. 微调后的轻量化的 Cycle GAN 模型的学习能力仍然有限，在初始下采样时采集的图像特征不够充足，模型在训练次数超过 50 次后，学习能力达到瓶颈；即使生成器、判别器的损失稳定且模型收敛，但转换后的图片更加偏向于真实风格。
3. 加入残差块的 Cycle GAN 模型的模型结构过于复杂，在残差网络之前采集了过多的特征，导致模型在训练时需要更高的 GPU 显存，对训练设备的要求高

- 4. 没有形成很好的衡量指标，本文尝试添加 FID（Fréchet Inception Distance）指标对转换后的图片质量进行定量分析，但实验过程中计算的 FID 指标均返回非数值 Nan；且本次实验时间有限，没有时间去进一步研究。
- 5. 本文在保存训练好的模型时操作失误，没有正确保存好模型，导致模型导出后不可以重新复用，无法进行 PSNR 等更多指标的测量。

最后，本文研究的 Cycle GAN 模型仍可以继续改进和进行性能提升，未来的展望主要如下：

- 1. 对于轻量化的 Cycle GAN 模型，增加生成器初始几次下采样的深度，提取更多原图的特征，使模型学习更多的图片特征，提高模型的图片风格转换能力。
- 2. 增加 FID、IS、PSNR 等图像质量指标等的计算，比对加入感知损失前后的轻量化的 Cycle GAN 模型对风格转换的效果。
- 3. 对于加入残差块的 Cycle GAN 模型，减少生成器的下采样次数，进一步简化模型的深度及复杂度，使得其训练所需的 GPU 显存更小；并在此基础上增加更多的模型训练次数，测试模型的收敛次数。
- 4. 将对于莫奈艺术风格准换的 Cycle GAN 模型扩展到其他艺术风格，并应用于实际生活中的风格转换（视频风格、滤镜等等）。

7 参考文献

[1]Zhu, J.-Y., Park, T., Isola, P., & Efros, A. A. (2017). Unpaired image-to-image translation using cycle-consistent adversarial networks. arXiv preprint arXiv:1703.10593.

[2] 古楚楚. 基于改进 CycleGAN 的图像艺术风格转换研究 [D]. 辽宁工程技术大学,2023.DOI:10.27210/d.cnki.glnju.2023.001152.

[3]高学秋,张祺,吴泽强,等.基于 MobileNetV2 的干电池底盖缺陷检测研究[J].信息技术与信息化,2024(03):166-169+174.

8 附录

轻量化的 Cycle GAN 模型

```
import numpy as np # linear algebra
from tensorflow.io.gfile import glob
import matplotlib.pyplot as plt
import tensorflow.keras as keras
import os
import tensorflow as tf
import tensorflow_probability as tfp

## Setting my path to the input file
#os.chdir("/kaggle/input/gan-getting-started")
for dirname, _, filenames in os.walk('/kaggle/input'):
    print(dirname, len(os.listdir(dirname)))
AUTOTUNE = tf.data.experimental.AUTOTUNE

if len(tf.config.list_physical_devices()) >= 3:
    strategy = tf.distribute.MirroredStrategy()
```

```

    print("Number of devices", strategy.num_replicas_in_sync)
else:
    strategy = tf.distribute.get_strategy()
strategy

# 加载图像数据

Monet_files = glob(str('/kaggle/input/monetdata/monet_tfrec/*.tfrec'))
print('Monet TFRecord Files:', len(Monet_files))
Photo_files = glob(str('/kaggle/input/monetdata/photo_tfrec/*.tfrec'))
print('Photo TFRecord Files:', len(Photo_files))

raw_dataset = tf.data.TFRecordDataset(Monet_files[0])
for raw_record in raw_dataset.take(1):
    example = tf.train.Example()
    example.ParseFromString(raw_record.numpy())
    Monet_features = [i for i in example.features.feature]

raw_dataset = tf.data.TFRecordDataset(Photo_files[0])
for raw_record in raw_dataset.take(1):
    example = tf.train.Example()
    example.ParseFromString(raw_record.numpy())
    Photo_features = [i for i in example.features.feature]

print('Monet tfrecord Features:', Monet_features)
print('Photo tfrecord Features:', Photo_features)

```

Load_dataset, 读取数据模块 输入的图像大小为(128X128X3)

```

import tensorflow as tf
from tensorflow.io.gfile import glob

# 设置图像的大小
IMAGE_SIZE = [128, 128]

# 编码图片为 (128, 128, 3)
def decode_image(image):
    image = tf.image.decode_jpeg(image, channels=3)
    image = tf.image.convert_image_dtype(image, tf.float32)
    image = tf.image.resize(image, IMAGE_SIZE)
    image = tf.reshape(image, [*IMAGE_SIZE, 3])
    return image

# 读取 TFRecord 文件
def read_tfrecord(example):
    tfrecord_format = {
        "image_name": tf.io.FixedLenFeature([], tf.string),
        "image": tf.io.FixedLenFeature([], tf.string),
        "target": tf.io.FixedLenFeature([], tf.string)
    }

```

```

    }
    example = tf.io.parse_single_example(example, tfrecord_format)
    image = decode_image(example['image'])
    return image

# 加载 TFRecord 文件中的数据集
def load_dataset(filenamees, labeled=True, ordered=False, repeat=False):
    dataset = tf.data.TFRecordDataset(filenamees)
    dataset = dataset.map(read_tfrecord, num_parallel_calls=AUTOTUNE)
    if repeat:
        dataset = dataset.repeat(count = 20)
    # 随机打乱样本
    dataset = dataset.shuffle(1000)
    # prefetch 训练时高效加载数据
    dataset = dataset.prefetch(buffer_size=tf.data.experimental.AUTOTUNE)
    return dataset

```

显示部分的原始真实图像、莫奈图像

```

# 每 32 张图片作为一个 batch
batch_size = 15
photo_ds = load_dataset(Photo_files, labeled=True).batch(batch_size)
monet_ds = load_dataset(Monet_files, labeled=True, repeat=True).batch(batch_size)

# 获取一个 batch 的数据
example_monet = next(iter(monet_ds))
File_example = glob(str('/kaggle/input/monetdata/test/*.tfrec'))
example_photo = load_dataset(File_example, labeled=True).batch(batch_size)
example_photo = next(iter(example_photo))

fig, axes = plt.subplots(nrows=2, ncols=5, figsize=(15, 6))
fig.suptitle('Image Categories', fontsize=16)

for i in range(0, 5):
    ax = axes[0, i]
    ax.imshow(example_monet[i])
    ax.axis('off')

for i in range(0, 5):
    ax = axes[1, i]
    ax.imshow(example_photo[i])
    ax.axis('off')

axes[0, 2].set_title('Monet', size='large', loc='center')
axes[1, 2].set_title('Photo', size='large', loc='center')
# 自动调整子图的间距
plt.tight_layout()
plt.show()

```

PCA 降维模块，主要用于起始状态的可视化

```

import tensorflow as tf
from sklearn.decomposition import PCA
# 将高维的特征图压缩为一维图像
def apply_pca_and_visualize(convolved_images):
    # Flatten the convolved images
    shape = convolved_images.shape
    normalized_image = convolved_images / 255.0
    reshaped_image = tf.reshape(normalized_image, (-1, shape[-1]))
    reshaped_array = reshaped_image.numpy()
    # Apply PCA
    pca = PCA(n_components=1)
    # 保留一个主成分的图片显示
    pca_result = pca.fit_transform(reshaped_array)
    # Reshape back
    pca_image_resized = pca_result.reshape(shape[0], shape[1], 1)
    return pca_image_resized

```

DownSample 下采样模块定义

```

import tensorflow as tf
import tensorflow.keras as keras
import tensorflow.keras.layers as layers
# 继承 keras.Layers.Layer 类
class Downsample(keras.layers.Layer):
    def __init__(self, filters, kernel_size=(2, 2), strides=(2, 2), padding='
same', activation='leaky_relu', **kwargs):
        super(Downsample, self).__init__(**kwargs)
        # 使用 tf.random_normal_initializer.RandomNormal 来随机初始化权重
        self.initializer = tf.random_normal_initializer(0., 0.02)
        self.gamma_init = keras.initializers.RandomNormal(mean=0.0, stddev=0.
02)
        # 初始化一些层, 包括 keras.Layers.Conv2D、keras.Layers.BatchNormalizatio
n 和 keras.Layers.Activation
        self.conv = layers.Conv2D(filters, kernel_size=kernel_size,
                                   strides=strides, padding=padding, kernel_init
ializer=self.initializer)
        self.bn = layers.BatchNormalization()
        self.activation = layers.Activation(activation=activation)

    # 定义前向前向传播过程
    def call(self, inputs, training=False):
        x = self.conv(inputs)
        x = self.bn(x, training=training)
        x = self.activation(x)
        return x

    # 返回所有配置参数的字典
    def get_config(self):
        config = super(Downsample, self).get_config()

```



```
# 设置优化器为 Adam 优化器, 损失函数为均方误差(MSE)
g.compile(optimizer='adam', loss='mse')
x = g(example_photo, training=False) ## to build the generator you need to pass an input through the network
g.summary()
```

Discriminator 判别器模块

```
import tensorflow as tf
import tensorflow.keras as keras
import tensorflow.keras.layers as layers
class Discriminator(keras.Model):
    def __init__(self, name='Discriminator', dense=32, activation='leaky_relu', **kwargs):
        super(Discriminator, self).__init__(name=name, **kwargs)
        self.initializer = tf.random_normal_initializer(0., 0.02)

        # 修改 Downsample 层的参数
        self.downsample1 = Downsample(32, name='Downsample_64') ## (bs, 64, 64, 32)
        self.downsample2 = Downsample(64, name='Downsample_32') ## (bs, 32, 32, 64)
        self.downsample3 = Downsample(128, name='Downsample_16') ## (bs, 16, 16, 128)
        self.downsample4 = Downsample(128, name='Downsample_8') ## (bs, 8, 8, 128)

        self.zeropad = layers.ZeroPadding2D(name='Zero_Padding')
        self.downsample5 = Downsample(256, strides=(1, 1), kernel_size=(2, 2),
                                         name='Downsample_stride_1') ## (bs, 8, 8, 256)

        self.flatten = layers.Flatten(name='Flatten_layer')

        # 调整全连接层 dense 的神经元数量
        self.dense = layers.Dense(dense, name=f'Hiden_layer_{dense}')
        self.dropout = layers.Dropout(0.2, name='Dropout')
        self.activation = layers.Activation(activation, name=f'{activation}')
        self.last = layers.Dense(1, activation='sigmoid')

    def call(self, inputs, training=False):
        x = self.downsample1(inputs, training=training)
        x = self.downsample2(x, training=training)
        x = self.downsample3(x, training=training)
        x = self.downsample4(x, training=training)
        x = self.zeropad(x)
        x = self.downsample5(x, training=training)
        x = self.flatten(x)
```

```

x = self.dense(x)
x = self.dropout(x)
x = self.activation(x)
x = self.last(x)
return x

```

Discriminator 判别器结构可视化

编译生成器

```

d = Discriminator()
d.compile(optimizer='adam', loss='mse')
y = d(example_monet, training=False)
d.summary()

```

Cycle GAN 网络模块

```

import tensorflow as tf
import tensorflow.keras as keras
import numpy as np
from tensorflow.keras.applications.inception_v3 import InceptionV3, preprocess_input
from tensorflow.keras.preprocessing.image import img_to_array, load_img
from tensorflow.keras.preprocessing import image
from scipy.linalg import sqrtm
# from tensorflow.keras.applications.vgg16 import VGG16
from tensorflow.keras.applications.mobilenet_v2 import MobileNetV2
# 创建CycleGAN 网络模型
class CycleGAN(keras.Model):
    def __init__(self, name='CycleGAN', lambda_cycle=10, lambda_identity=5, trainable=False, dtype='float32'):
        super(CycleGAN, self).__init__(name=name, trainable=trainable, dtype=dtype)

        self.monet_generator_ = Generator(name='Monet_Generator')
        self.monet_discriminator_ = Discriminator(name='Monet_Discriminator')
        self.photo_generator_ = Generator(name='Photo_Generator')
        self.photo_discriminator_ = Discriminator(name='Photo_Discriminator')
        self.lambda_cycle = lambda_cycle
        self.lambda_identity = lambda_identity
        # 加载预训练的 VGG16 模型
        # self.vgg = VGG16(weights='imagenet', include_top=False, input_shape=(256, 256, 3))
        # self.vgg.trainable = False
        self.mobilenet = MobileNetV2(weights='imagenet', include_top=False, input_shape=(128, 128, 3))
        self.mobilenet.trainable = False

    def compile(self, m_gen_optimizer, p_gen_optimizer, m_disc_optimizer, p_disc_optimizer, **kwargs):
        super(CycleGAN, self).compile(**kwargs)
        # 初始化生成器、判别器m的优化器

```

```

self.m_gen_optimizer = m_gen_optimizer
self.m_disc_optimizer = m_disc_optimizer
# 初始化生成器、判别器p的优化器
self.p_gen_optimizer = p_gen_optimizer
self.p_disc_optimizer = p_disc_optimizer
# 二分类交叉熵
self.bce_ = tf.keras.losses.BinaryCrossentropy(from_logits=False)

# 生成器损失
def generator_loss_(self, disc_output):
    return self.bce_(tf.ones_like(disc_output), disc_output)

# 判别器损失
def discriminator_loss_(self, real_output, fake_output):
    real_loss = self.bce_(tf.ones_like(real_output), real_output)
    fake_loss = self.bce_(tf.zeros_like(fake_output), fake_output)
    return (real_loss + fake_loss)*0.5

# 循环网络损失
def cycle_loss_(self, real_image, cycled_image):
    return self.lambda_cycle * tf.reduce_mean(tf.abs(real_image - cycled_image))

# 身份损失
def identity_loss_(self, real_image, same_image):
    return self.lambda_identity * tf.reduce_mean(tf.abs(real_image - same_image))

# 计算感知损失
def perceptual_loss_(self, real_image, generated_image):
    # 通过vgg预训练网络计算L1 loss
    real_features = self.mobilenet(real_image)
    generated_features = self.mobilenet(generated_image)
    loss = tf.reduce_mean(tf.abs(real_features - generated_features))
    return loss

# 训练过程
def train_step(self, batch_data):
    real_monet, real_photo = batch_data
    ### Persistent True to compute multiple gradients for the same tape
    with tf.GradientTape(persistent=True) as tape:
        # 前向传播
        fake_monet = self.monet_generator_(real_photo, training=True) ##
        # Photo to fake Monet
        cycled_photo = self.photo_generator_(fake_monet, training=True) #
        # fake Monet to photo

```

```

        fake_photo = self.photo_generator_(real_monet, training=True) ##
Monet to fake photo
        cycled_monet = self.monet_generator_(fake_photo, training=True) #
# fake Photo to Monet

        ### Identity mapping (G(x) to x, F(y) to y)
        same_monet = self.monet_generator_(real_monet, training=True) ##
Monet to monet
        same_photo = self.photo_generator_(real_photo, training=True) ##
Photo to photo

        # 判别器预测, 4 个判别器
        disc_real_monet = self.monet_discriminator_(real_monet, training=
True) ## Monet is real Monet
        disc_fake_monet = self.monet_discriminator_(fake_monet, training=
True) ## Fake Monet is real Monet
        disc_real_photo = self.photo_discriminator_(real_photo, training=
True) ## Photo is a real photo
        disc_fake_photo = self.photo_discriminator_(fake_photo, training=
True) ## Fake Photo is a real photo

        # 计算损失
        ## Generators
        gen_monet_loss = self.generator_loss_(disc_fake_monet)
        gen_photo_loss = self.generator_loss_(disc_fake_photo)
        ## Discriminators
        disc_monet_loss = self.discriminator_loss_(disc_real_monet, disc_
fake_monet)
        disc_photo_loss = self.discriminator_loss_(disc_real_photo, disc_
fake_photo)

        ### Cycle
        cycle_loss = self.cycle_loss_(real_monet, cycled_monet) + self.cyc
le_loss_(real_photo, cycled_photo)
        ### Identity
        identity_loss = self.identity_loss_(real_monet, same_monet) + sel
f.identity_loss_(real_photo, same_photo)

        ### Perceptual
        perceptual_loss_monet = self.perceptual_loss_(real_monet, fake_mo
net)
        perceptual_loss_photo = self.perceptual_loss_(real_photo, fake_ph
oto)

#         ### Generators total Loss
#         total_gen_monet_loss = gen_monet_loss + cycle_loss + identity_lo
ss

```



```

#         total_gen_photo_loss = gen_photo_loss + cycle_loss + identity_lo
ss

        # perceptual_loss
        total_gen_monet_loss = gen_monet_loss + cycle_loss + identity_loss
+ perceptual_loss_monet
        total_gen_photo_loss = gen_photo_loss + cycle_loss + identity_loss
+ perceptual_loss_photo

        # Gradients 梯度
        # 计算对应损失函数以及模型可训练参数（卷积层、批归一化层、激活函数等网络层的
权重和偏置）
        monet_generator_gradients = tape.gradient(total_gen_monet_loss, self.
monet_generator_.trainable_variables)
        photo_generator_gradients = tape.gradient(total_gen_photo_loss, self.
photo_generator_.trainable_variables)
        monet_discriminator_gradients = tape.gradient(disc_monet_loss, self.m
onet_discriminator_.trainable_variables)
        photo_discriminator_gradients = tape.gradient(disc_photo_loss, self.p
hoto_discriminator_.trainable_variables)

        # gradients 梯度的反向传播
        # 使用zip 将梯度和训练参数打包，并对训练参数进行梯度下降操作
        self.m_gen_optimizer.apply_gradients(zip(monet_generator_gradients, s
elf.monet_generator_.trainable_variables))
        self.p_gen_optimizer.apply_gradients(zip(photo_generator_gradients, s
elf.photo_generator_.trainable_variables))
        self.m_disc_optimizer.apply_gradients(
            zip(monet_discriminator_gradients, self.monet_discriminator_.trai
nable_variables))
        self.p_disc_optimizer.apply_gradients(
            zip(photo_discriminator_gradients, self.photo_discriminator_.trai
nable_variables))

        # #
        # # 计算FID
        # fid_score = self.calculate_fid(real_monet, fake_monet) + self.calcu
late_fid(real_photo, fake_photo)
        # # 计算IS
        # is_score = self.calculate_is(fake_monet) + self.calculate_is(fake_p
hoto)

        return {
            "monet_gen_loss": total_gen_monet_loss,
            "photo_gen_loss": total_gen_photo_loss,
            "monet_disc_loss": disc_monet_loss,
            "photo_disc_loss": disc_photo_loss

```

```
}
```

用于训练过程的可视化的 **callback**

```
class callbacks(keras.callbacks.Callback):
    def __init__(self, fid_interval, generator, example_photo, example_monet):
        self.fid_interval = fid_interval
        self.generator = generator
        self.example_photo = example_photo
        self.example_monet = example_monet
        self.fid_values_monet = []
        self.fid_values_photo = []
        self.losses = []

    def on_epoch_end(self, epoch, logs={}):
        ### Every 5 epochs plot the generator
        randomint = np.random.randint(0, 15, 1)[0]

        # 加入损失
        self.losses.append(logs)
        # print(epoch, logs)

        if epoch % self.fid_interval == 0:
            fake_monet = self.generator.monet_generator_(self.example_photo,
training=True)
            fake_photo = self.generator.photo_generator_(self.example_monet,
training=True)
            fid_monet_value = self.calculate_fid(fake_monet, self.example_monet)
            fid_photo_value = self.calculate_fid(fake_photo, self.example_photo)

            print(f"fid_monet:{fid_monet_value},fid_photo:{fid_photo_value}")
            self.fid_values_monet.append(fid_monet_value)
            self.fid_values_photo.append(fid_photo_value)
            # save model
            tf.saved_model.save(cycle_gan, f'mymodel/{epoch}/')

        if epoch != 0:
            fig, axes = plt.subplots(nrows=1, ncols=4, figsize=(15, 4))
            fig.suptitle(f'Generator evolution epoch #{epoch}', fontsize=16)
            ax = axes[0]
            ax.imshow(example_photo[randomint])
            axes[0].set_title('Original Photo', size='large', loc='center')
            ax.axis('off')
            ax = axes[1]
            ax.imshow(cycle_gan.monet_generator_(example_photo[randomint])
```

```

axes[1].set_title('Photo to Monet', size='large', loc='center')
ax.axis('off')
ax = axes[2]
ax.imshow(example_monet[randomint])
axes[2].set_title('Original Monet', size='large', loc='center')
ax.axis('off')
ax = axes[3]
ax.imshow(cycle_gan.photo_generator_(example_monet)[randomint])
axes[3].set_title('Monet to photo', size='large', loc='center')
ax.axis('off')
plt.savefig(f'{epoch}.png')
plt.show()
# save model
# 保存模型
tf.saved_model.save(cycle_gan, f'mymodel/{epoch}/')

if epoch == 0:
    fig, axes = plt.subplots(nrows=1, ncols=4, figsize=(15, 4))
    fig.suptitle(f'Starting point', fontsize=16)
    ax = axes[0]
    ax.imshow(example_photo[randomint])
    axes[0].set_title('Original Photo', size='large', loc='center')
    ax.axis('off')
    ax = axes[1]
    ax.imshow(cycle_gan.monet_generator_(example_photo)[randomint])
    axes[1].set_title('Photo to Monet', size='large', loc='center')
    ax.axis('off')
    ax = axes[2]
    ax.imshow(example_monet[randomint])
    axes[2].set_title('Original Monet', size='large', loc='center')
    ax.axis('off')
    ax = axes[3]
    ax.imshow(cycle_gan.photo_generator_(example_monet)[randomint])
    axes[3].set_title('Monet to photo', size='large', loc='center')
    ax.axis('off')
    plt.savefig(f'{epoch}.png')
    plt.show()

def calculate_fid(self, real_images, fake_images):
    # 调整图像大小为299x299
    real_images = tf.image.resize(real_images, (299, 299))
    fake_images = tf.image.resize(fake_images, (299, 299))

    # 图像预处理
    real_images = tf.keras.applications.inception_v3.preprocess_input(real_images)

```

```

fake_images = tf.keras.applications.inception_v3.preprocess_input(fake_images)

# 加载 InceptionV3 模型
model = tf.keras.applications.InceptionV3(include_top=False, pooling='avg', input_shape=(299, 299, 3))

# 提取特征
real_features = model(real_images)
fake_features = model(fake_images)

# 计算 FID
mu_real = tf.reduce_mean(real_features, axis=0)
mu_fake = tf.reduce_mean(fake_features, axis=0)

# 计算样本协方差矩阵
cov_real = tfp.stats.covariance(real_features, sample_axis=0)
cov_fake = tfp.stats.covariance(fake_features, sample_axis=0)

diff = mu_real - mu_fake
# 使用数值稳定的方法计算 FID
cov_sqrtm = tf.linalg.sqrtm(tf.linalg.matmul(cov_real, cov_fake))
trace = tf.linalg.trace(cov_real + cov_fake - 2 * cov_sqrtm)
fid = tf.sqrt(tf.reduce_sum(diff ** 2) + tf.clip_by_value(trace, 0, 1e9))

return fid

```

使用 **Adam** 优化器，创建 **CycleGAN** 模型并打印信息

```

initial_learning_rate = 2e-3 # 初始学习率为 2e-3
decay_steps = 800
decay_rate = 0.1 # 将衰减率从 0.96 改为 0.1
with strategy.scope():
    # Learning rate schedules for each optimizer
    monet_gen_lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
        initial_learning_rate, decay_steps, decay_rate, staircase=True)

    photo_gen_lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
        initial_learning_rate, decay_steps, decay_rate, staircase=True)

    monet_disc_lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
        initial_learning_rate, decay_steps, decay_rate, staircase=True)

    photo_disc_lr_schedule = tf.keras.optimizers.schedules.ExponentialDecay(
        initial_learning_rate, decay_steps, decay_rate, staircase=True)
    m_gen_opt = tf.keras.optimizers.Adam(learning_rate=monet_gen_lr_schedule,
        beta_1=0.5)

```

```

    p_gen_opt = tf.keras.optimizers.Adam(learning_rate=photo_gen_lr_schedule,
    beta_1=0.5)
    m_disc_opt = tf.keras.optimizers.Adam(learning_rate=monet_disc_lr_schedul
e, beta_1=0.5)
    p_disc_opt = tf.keras.optimizers.Adam(learning_rate=photo_disc_lr_schedul
e, beta_1=0.5)
# # 使用Adam 优化器, 学习率为2e-3, beta_1 为0.5
# with strategy.scope():
#     m_gen_opt = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
#     p_gen_opt = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
#     m_disc_opt = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
#     p_disc_opt = tf.keras.optimizers.Adam(2e-4, beta_1=0.5)
# 创建CycleGAN 模型并打印信息
with strategy.scope():
    cycle_gan = CycleGAN()
    cycle_gan.compile(m_gen_opt, p_gen_opt, m_disc_opt, p_disc_opt)
cycle_gan.summary()

```

开始训练 Cycle GAN 模型, 并对训练结果进行可视化

```

# 创建回调函数
fid_interval = 5 # 每隔5 个epoch 计算一次FID
callback = callbacks(fid_interval, cycle_gan, example_photo, example_monet)

# 训练模型
cycle_gan.fit(tf.data.Dataset.zip((monet_ds, photo_ds)), epochs=100, verbose
=1, callbacks=callback)

# 对部分的图片进行莫奈风格转换后显示
fig, axes = plt.subplots(nrows=4, ncols=5, figsize=(15, 6))
fig.suptitle('Results', fontsize=16)

monet_results = cycle_gan.monet_generator_(example_photo)[0:10]
# 显示原始照片
for i in range(10):
    ax = axes[i // 5, i % 5]
    ax.imshow(example_photo[i])
    ax.axis('off')

# 显示莫奈风格的图像
for count, element in enumerate(monet_results):
    ax = axes[2 + count // 5, count % 5]
    ax.imshow(element)
    ax.axis('off')

axes[0, 2].set_title('Photos', size='large', loc='center')
axes[2, 2].set_title('Photos as Monet', size='large', loc='center')
plt.tight_layout() # Adjust layout to make room for the main title
plt.savefig(f'final.png')

```



```
plt.show()
```

损失函数的可视化

可视化损失函数

```
gen_monet_losses = [epoch['monet_gen_loss'] for epoch in callback.losses]
photo_gen_losses = [epoch['photo_gen_loss'] for epoch in callback.losses]
monet_disc_losses = [epoch['monet_disc_loss'] for epoch in callback.losses]
photo_disc_losses = [epoch['photo_disc_loss'] for epoch in callback.losses]
```

```
epochs = range(1, len(callback.losses) + 1)
```

```
plt.figure(figsize=(10, 5))
plt.plot(epochs, gen_monet_losses)
plt.plot(epochs, photo_gen_losses)
plt.plot(epochs, monet_disc_losses)
plt.plot(epochs, photo_disc_losses)
plt.scatter(epochs, gen_monet_losses, label='Monet Generator Loss')
plt.scatter(epochs, photo_gen_losses, label='Photo Generator Loss')
plt.scatter(epochs, monet_disc_losses, label='Monet Discriminator Loss')
plt.scatter(epochs, photo_disc_losses, label='Photo Discriminator Loss')
plt.title('CycleGAN Training Losses')
plt.xlabel('Epoch')
plt.ylabel('Loss')
plt.legend()
plt.savefig("loss.png")
plt.show()
```

可视化 FID 和 IS 指标

创建一个新的图像

```
plt.figure(figsize=(10, 6))
epochs = range(1, len(callback.losses) + 1, 5)
```

绘制莫奈风格图像的 FID 值

```
plt.plot(epochs, callback.fid_values_monet, label='Monet Style')
plt.scatter(epochs, callback.fid_values_monet, label='Monet Style')
```

绘制照片的 FID 值

```
plt.plot(epochs, callback.fid_values_photo, label='Photo Style')
plt.scatter(epochs, callback.fid_values_photo, label='Photo Style')
```

```
plt.title('CycleGAN FID Score')
plt.xlabel('Epoch')
plt.ylabel('Score')
plt.legend()
plt.savefig("FID_Score.png")
plt.show()
```

保存模型

```
tf.saved_model.save(cycle_gan, 'mymodel/')
```

加入残差块的 Cycle GAN 模型

该部分的加载数据部分、下采样、上采样、网络定义、可视化部分都和轻量化的 Cycle GAN 模型相同，不多赘述

Residual Block

```
class ResidualBlock(keras.layers.Layer):
    def __init__(self, filters = 256, kernel_size=(3, 3), strides=(1, 1), padding='same', activation = 'leaky_relu', **kwargs):
        super(ResidualBlock, self).__init__(**kwargs)
        self.conv1 = layers.Conv2D(filters, kernel_size, strides, padding)
        self.bn1 = layers.BatchNormalization()
        self.activation = layers.ReLU()
        self.conv2 = layers.Conv2D(filters, kernel_size, strides, padding)
        self.bn2 = layers.BatchNormalization()

    def call(self, inputs, training=False):
        x = self.conv1(inputs)
        x = self.bn1(x, training=training)
        x = self.activation(x)
        x = self.conv2(x)
        x = self.bn2(x)
        return layers.Add()([inputs, x])
```

Generator 定义

```
class Generator(keras.Model):
    def __init__(self, name = 'Generator', **kwargs):
        super(Generator, self).__init__(name = name, **kwargs)

        ### Downsampling Layers
        self.downsample1 = Downsample(64, name = 'Downsample_same', strides = (1,1), kernel_size = (5,5)) ## (bs,256,256,64)
        self.downsample2 = Downsample(128, name = 'Downsample_128') ## (bs,128,128,128)
        self.downsample3 = Downsample(128, name = 'Downsample_64') ## (bs,64,64,128)
        self.downsample4 = Downsample(256, name = 'Downsample_32') ## (bs,32,32,256)
        self.downsample5 = Downsample(512, name = 'Downsample_16') ## (bs,16,16,512)
        self.downsample6 = Downsample(512, name = 'Downsample_8') ## (bs,8,8,512)
        self.downsample7 = Downsample(512, name = 'Downsample_4') ## (bs,4,4,512)

        self.residual1 = ResidualBlock()
        self.residual2 = ResidualBlock()
        self.residual3 = ResidualBlock()
```

```

    ## Upsampling layers
    self.upsample1 = Upsample(512, name = 'Upsample_8') ## (bs,8,8,512)
    self.upsample2 = Upsample(512, name = 'Upsample_16') ## (bs,16,16,51
2)
    self.upsample3 = Upsample(256,name = 'Upsample_32' ) ## (bs,32,32,51
2)
    self.upsample4 = Upsample(128, name = 'Upsample_64') ## (bs,64,64,25
6)
    self.upsample5 = Upsample(128, name = 'Upsample_128') ## (bs,128,128,
128)
    self.upsample6 = Upsample(64, name = 'Upsample_256') ## (bs,256,256,6
4)
    self.final_upsample = Upsample(3, activation = 'tanh', name = 'Output
_layer', strides = (1,1)) # (bs,256,256,3)

def call(self, inputs, training = False):
    ### Downsampling
    d1 = self.downsample1(inputs, training = training)
    d2 = self.downsample2(d1, training = training)
    d3 = self.downsample3(d2, training = training)
    d4 = self.downsample4(d3, training = training)
    d5 = self.downsample5(d4, training = training)
    d6 = self.downsample6(d5, training = training)
    d7 = self.downsample7(d6, training = training)

    r1 = self.residual1(d7, training = training)
    r2 = self.residual2(r1, training = training)
    r3 = self.residual3(r2, training = training)

    ## Upsampling
    u1 = self.upsample1(r3, training = training)
    u1_concat = tf.concat([u1, d6], axis=-1)
    u2 = self.upsample2(u1_concat, training = training)
    u2_concat = tf.concat([u2, d5], axis=-1)
    u3 = self.upsample3(u2_concat, training = training)
    u3_concat = tf.concat([u3, d4], axis=-1)
    u4 = self.upsample4(u3_concat, training = training)
    u4_concat = tf.concat([u4, d3], axis=-1)
    u5 = self.upsample5(u4_concat, training = training)
    u5_concat = tf.concat([u5, d2], axis=-1)
    u6 = self.upsample6(u5_concat, training = training)
    u6_concat = tf.concat([u6, d1], axis=-1)
    x = self.final_upsample(u6_concat, training = training)
    return x

```

Discriminator 定义

```
class Discriminator(keras.Model):
    def __init__(self, name = 'Discriminator', dense = 64, activation = 'leaky_relu', **kwargs):
        super(Discriminator, self).__init__(name = name, **kwargs)
        self.initializer = tf.random_normal_initializer(0., 0.02)
        self.downsample1 = Downsample(64, name = 'Downsample_128') ## (bs, 128, 128, 64)
        self.downsample2 = Downsample(128, name = 'Downsample_64') ## (bs, 64, 64, 128)
        self.downsample3 = Downsample(256, name = 'Downsample_32') ## (bs, 32, 32, 256)
        self.downsample4 = Downsample(256, name = 'Downsample_16') ## (bs, 32, 32, 256)
        self.zeropad = layers.ZeroPadding2D(name = 'Zero_Padding')
        self.downsample5 = Downsample(512, strides = (1,1), kernel_size = (2, 2), name = 'Downsample_stride_1') ## (bs, 32, 32, 512)
        self.flatten = layers.Flatten(name = 'Flatten_layer')
        self.dense = layers.Dense(dense, name = f'Hiden_layer_{dense}')
        self.dropout = layers.Dropout(0.2, name = 'Dropout')
        self.activation = layers.Activation(activation, name = f'{activation}')
    )
    self.last = layers.Dense(1, activation = 'sigmoid')

    def call(self, inputs, training = False):
        x = self.downsample1(inputs, training = training)
        x = self.downsample2(x, training = training)
        x = self.downsample3(x, training = training)
        x = self.downsample4(x, training = training)
        x = self.zeropad(x)
        x = self.downsample5(x, training = training)
        x = self.flatten(x)
        x = self.dense(x)
        x = self.dropout(x)
        x = self.activation(x)
        x = self.last(x)
        return x
```